PipedPeer: Zero-Configuration Distributed Computing via Recursive, Decentralised Coordination

submitted in partial fulfillment of the requirement
for the award of the Degree of

Bachelor of Technology

in

Computer Engineering

by

Yateen Vaviya (2023300251)

Vinayak Yadav (2023300264)

Sahil Gupta (2023300072)

under the guidance of

Prof. Rupali Sawant

Department of Computer Engineering

Bharatiya Vidya Bhavan's

Sardar Patel Institute of Technology

(Autonomous Institute Affiliated to University of Mumbai)

Munshi Nagar, Andheri-West, Mumbai-400058

University of Mumbai

2026–2027

Approval Certificate

This is to certify that the project entitled **PipedPeer: Zero-Configuration Distributed Computing via Recursive, Decentralised Coordination** by Yateen Vaviya, Vinayak Yadav and Sahil Gupta is approved for the partial fulfillment of the Major Project for the Final Year Project towards obtaining the Degree of Bachelor of Technology in Computer Engineering from University of Mumbai.

Project Guide	External Examiner
Name: Prof. Rupali Sawant	Name: _____
Date: _____	Date: _____
Head of Department	Principal

Seal of the Institute

Declaration

We declare that the work titled **PipedPeer: Zero-Configuration Distributed Computing via Recursive, Decentralised Coordination** is our own work carried out under the guidance of **Prof. Rupali Sawant** at Sardar Patel Institute of Technology. This work has not been submitted elsewhere for any degree.

Yateen Vaviya	**Vinayak Yadav**	**Sahil Gupta**
2023300251	**2023300264**	**2023300072**

ABSTRACT

This project presents **PipedPeer**, a system that runs an unmodified Python script on other machines, with no cluster to configure and no manual installation: one binary, whose set-up command installs the rest (Nix and an OCI runtime) if absent, without root if need be. Three problems normally stand between a developer and the idle machines around them: reproducing the script's environment elsewhere, rewriting the program against a cluster API, and reaching machines that sit behind different routers. PipedPeer removes all three.

The environment is reproduced by building a **Nix closure** against a pinned package set, so the same script yields the same content-addressed store path on every machine and on any day. A worker that already holds an environment pays nothing to set it up again, and one that holds part of it receives only the store paths it lacks. Measured on a three-worker container lab, the first submission of a job took 23.4 seconds, and later submissions that found the environment already in place took a median of 0.83 seconds.

The program is distributed by **intercepting the parallel primitives it already uses**. A shim injected into the job's interpreter replaces `multiprocessing.Pool`, pandas `groupby` and `merge`, chunked file readers, NumPy linear algebra, `joblib` and torch data-parallel training. Pool kernels travel as source or, for lambdas and closures, by value through cloudpickle. Work is shared between machines in proportion to their measured throughput, and a cost model ships numerical work only when computing locally would cost more than moving it. Every remote path degrades to the local one, with one deliberate exception: a matrix product that fits neither locally nor on the cluster raises an error rather than exhausting the machine.

Machines on different networks are joined with one command. A small introducer exchanges candidate addresses; the machines then ask their routers for a port mapping or punch through them, falling back to a birthday-collision search for the hardest pair of NATs, and carry all traffic over a direct, mutually authenticated QUIC link. No relay carries job data. Each daemon's API is protected by an optional shared secret and pinned TLS, closures are signed with each node's ed25519 key, and jobs run by default in a `crun` sandbox under their own memory ceiling where the node supports it. A run can be watched (a plain-language events feed, CPU and GPU load per node, a closing summary of where its work ran), checked beforehand by a dry run, stopped everywhere with one interrupt, or detached and reattached later; a training ring continues when a rank dies and takes back a machine that returns. Of **829** tests, **822** pass, **0** fail and **7** are skipped; a mutation audit that breaks 95 features one at a time is caught by the tests in every case. A fault-injection run on the container lab did not distribute work and so did

not test recovery, and is reported as such. Speedups between real machines are reported only as recorded during development, with their provenance, and are not claimed beyond them.

Keywords: Distributed Computing, Peer-to-Peer, NAT Traversal, Transparent Interception, Nix, Content-Addressed Caching, OCI Sandboxing, Cost-Based Scheduling, Fault Tolerance.

Contents

1 Introduction **11**

1.1 Problem Statement . . . 11
1.2 Objectives . . . 12
1.3 Scope . . . 12
1.4 Technologies Used . . . 14
1.5 Assumptions and Constraints . . . 14

2 Literature Survey **16**

Paper 1 . . . 16
Paper 2 . . . 16
Paper 3 . . . 17
Paper 4 . . . 17
Paper 5 . . . 17
Paper 6 . . . 18
Paper 7 . . . 18
Paper 8 . . . 18
Paper 9 . . . 19
Paper 10 . . . 19
Paper 11 . . . 19
Paper 12 . . . 20
Paper 13 . . . 20
Paper 14 . . . 20
Paper 15 . . . 21
Paper 16 . . . 21

Standards Underlying Cross-Network Operation . . . 21

3 Analysis **23**

3.1 System Architecture . . . 23
3.2 Data Flow Diagram . . . 25
Level 0 — Context Diagram . . . 25
Level 1 — System Decomposition . . . 27
3.3 Use Case Diagram . . . 29

3.4 Sequence Diagram 30

4 Design and Methodology **32**
4.1 Node Discovery on a Local Network 32
4.2 Joining Across Networks 32
4.3 Security and Identity 35
4.4 Placement and Scoring 37
4.5 The Lease Protocol 39
4.6 Reproducing the Environment 40
4.7 Closure Transfer and Cache Reuse 42
4.8 Execution and Isolation 44
4.9 Transparent Interception 47
4.10 The Cost Model 52
4.11 Fault Tolerance 54
4.12 Resource Estimation 55
4.13 Seeing What the Cluster Does 55
4.14 Test Workloads 56

5 Results and Discussion **57**
5.1 Test Bed and Sources of Evidence 57
5.2 Functional Verification 58
5.3 Environment Cache 59
5.4 The Cost of Leaving Interception On 60
5.5 Work Crosses Node Boundaries 62
5.6 Fault Tolerance 62
5.7 Sandbox Cost 63
5.8 Measurements Between Real Machines 63
5.9 Scale of the Implementation 65
5.10 Rehearsing the Demonstration 65
5.11 Comparative Analysis 68
5.12 What Was Not Measured 69

6 Conclusion and Future Work **71**
6.1 Conclusion 71
6.2 Limitations of the Current System 72
6.3 Security 74
6.4 Future Work 74

List of Figures

3.1 PipedPeer system design. The submitting side builds the environment and places the job; the worker admits it, caches the closure and runs it sandboxed under a memory ceiling. Both roles are the same binary. Peers on other networks are reached over direct QUIC links that an introducer helps establish but never carries. 24

3.2 Level 0 DFD — context diagram. The introducer and the registry are dashed because the system works without them. 26

3.3 Level 1 DFD — seven processes over four data stores. 27

3.4 Use cases. A peer daemon is an actor in its own right, because every node serves closures and executes chunks for others; the introducer registers nodes and forwards punch requests. 29

3.5 End-to-end execution of one job. 30

4.1 Joining across networks. The introducer exchanges candidate addresses and forwards one punch request; the two nodes then race their candidate pairs over a single UDP socket, fall back to a birthday collision when one NAT allocates a port per destination, and carry everything over a mutually authenticated QUIC link exposed locally as an ordinary node. 33

4.2 The three layers of trust: a shared secret on every API route, pinned TLS (or QUIC across networks) on the wire, and the node key that both authenticates links and signs closures. 35

4.3 Placement: gather, filter, score, lease, retry. A task is never dropped; when nothing is eligible it is queued and retried. 37

4.4 Left: the range each term can contribute. Right: the same function applied to an idle strong node and a loaded weak one. 38

4.5 Lease states. Recovery is a consequence of expiry rather than a separate mechanism. 39

4.6 The store path is the cluster-wide cache key. Most submissions ship no environment at all, and a related environment ships only the store paths the receiver lacks. 42

4.7 What the job can and cannot see. The absences are as deliberate as the mounts. Shared memory is mounted world-writable because Python's `multiprocessing` requires it, and the driver libraries are bound file by file so that the host C library cannot shadow the closure's. 44
4.8 How the shim reaches the interpreter and what it replaces. 47
4.9 Hash shuffle. Correctness follows from key locality, not from combiner algebra. 50
4.10 Gradient exchange over the daemon port. The lead daemon reduces the sample-weighted gradients and returns one average. 52
4.11 Decision regions of Equation 4.2 for the pandas path ($\rho = 1$, $R = R' = 1\,\text{GFLOP/s}$) at three nodes on a 500 MB/s link. Grouped aggregation and joins have low arithmetic intensity, so they are distributed only above 512 MB. The NumPy path uses the same inequality with its own throughputs and is not drawn. 53
4.12 Racing local and remote work. The local side is always able to finish alone, which is what makes the guarantee unconditional. 54

5.1 Left: first submission against the median of the submissions that sent nothing. Right: every run, coloured by what its log says was sent. 60
5.2 Left: the parallel-map job of Table 5.4, clean runs only (failed-and-retried runs excluded). Right: interpreter start-up, with eager patching (August) and deferred patching (October). 62
5.3 Sandbox start cost, median of 20 iterations (August 2026). 63
5.4 The same comparison in matrix form. The criteria in the upper block were chosen because they are what PipedPeer set out to change; the lower block is included so the table is not simply a list of its own features. 69

List of Tables

1.1 Technologies used in PipedPeer 14

3.1 Subsystems and their responsibilities 25

4.1 Intercepted primitives and their distribution strategies 49

5.1 Test suite results (October 2026) 58
5.2 Examples of what the tests establish 59
5.3 Five submissions of one environment (October 2026) 59
5.4 Whole-job time of a 48-item parallel map with interception enabled and disabled (October 2026, five runs each) 60
5.5 Whole-job time of a declined 16-item parallel map (12 October 2026, second host, five runs each) 61
5.6 Interpreter start-up overhead for a job that imports nothing heavy 61
5.7 Recorded measurements between real machines (August 2026) 64
5.8 Implementation size at commit `4c91ef3` 65
5.9 Demonstration steps, first run and cached (12 October 2026, container labs on one laptop) 66
5.10 Failure drills in the rehearsal 67
5.11 PipedPeer against existing systems. The lower block is where PipedPeer is partial or the weaker system. 68
5.12 Measurements not made for this report, and why 70

Chapter 1

Introduction

1.1 Problem Statement

A developer with a laptop, a lab workstation and a spare desktop has a considerable amount of idle computing capacity and no practical way to use it. The frameworks that exist for distributed computation — Apache Spark [1], Ray [2], Dask [4] — all presuppose a cluster that somebody has already configured: a head node, matching software on every worker, a scheduler process, and a network on which every machine can reach every other. Ordinary machines fail the last of these: each sits behind its own home or campus router and accepts no inbound connection. For a team that simply wants a script to finish sooner, that setup costs more than the job is worth. The alternative, `ssh` together with `rsync`, means reproducing the environment by hand on every machine and receiving no scheduling, no isolation and no fault tolerance in return.

Three distinct problems sit underneath this, and they are usually attacked separately:

1. **The environment problem.** A script needs a particular interpreter and a particular set of libraries. Making a remote machine match the local one is the bulk of the work, and it is the reason "just `ssh` into it" does not scale past a single machine.

2. **The code problem.** Even with a matching environment, using more than one machine normally means rewriting the program against a cluster API. A script that took an afternoon to write becomes a project.

3. **The reachability problem.** Machines on different networks cannot open a connection to one another without a VPN, port forwarding, or a relay that carries every byte between them.

PipedPeer: Zero-Configuration Distributed Computing via Recursive, Decentralised Coordination addresses all three without asking the user to change the program. The environment is reproduced exactly by shipping a Nix closure; the program is distributed without modification by intercepting the parallel primitives it already uses; and machines on different networks are joined by punching a direct path through their routers, with a small introducer that only tells each machine where the others are.

1.2 Objectives

1. To run an unmodified Python script on the best available machine on the network, with results appearing in the user's working directory as though the script had run locally.
2. To reproduce the script's environment exactly on that machine, sending only the store paths that machine does not already hold.
3. To choose the machine from live capacity, treating architecture and GPU availability as hard requirements and load, memory and hardware as a ranking.
4. To distribute the parallel work inside the script across the cluster with no code changes, only when doing so is expected to be faster than running it locally, and in proportion to each machine's measured throughput.
5. To join machines on different networks into one cluster with a single command, with no VPN, no port forwarding and no relay in the data path.
6. To protect each node's API with a shared secret and its traffic with encryption, to isolate each job, and to survive the loss of any worker without losing the run.
7. To require no manual installation on a new machine: one binary, whose set-up command installs what else is needed, without root if necessary.

1.3 Scope

The following are within scope and implemented:

- A Go daemon that performs peer discovery, placement, leasing, sandboxed execution and result delivery, and a thin CLI client.
- Peer discovery by UDP multicast and broadcast on the local subnet, with a TCP health-probe sweep as a fallback and an optional standalone registry.
- Joining across networks: an introducer, router port mapping (PCP, NAT-PMP, UPnP-IGD), UDP hole punching with a birthday-collision fallback, and direct QUIC links authenticated by each node's ed25519 key.
- An optional cluster-wide shared secret on every API route, pinned TLS on the daemon port, and closures signed with each node's key.
- Reproducible environments built as Nix closures against a pinned package set, cached by store path, and transferred as a path-level difference against what the receiving node already holds.

- Placement by hard filters on architecture, GPU, memory, cores and VRAM, followed by a score over live load and hardware, with a three-phase lease.
- Per-job isolation in a `crun` OCI sandbox under a per-job memory ceiling, including GPU device and driver passthrough, runnable without root.
- Transparent interception of `multiprocessing.Pool`, pandas `groupby`, `merge` and chunked readers, NumPy linear algebra, `joblib` and scikit-learn, and torch data-parallel training, with work shared by measured throughput and numerical work governed by a cost model.
- Result synchronisation of changed files only, with deletions carried in a manifest, and a live terminal dashboard.

The following are explicitly out of scope in the current implementation: per-user authorisation (a cluster shares one secret; Section 6.3), runtimes other than Python, relaying traffic for a peer that no direct path can reach (such a peer is reported and routed around), and any economic incentive mechanism.

1.4 Technologies Used

Table 1.1: Technologies used in PipedPeer

Technology	**Category**	**Purpose**
Go 1.25	Core language	Daemon, CLI and coordinator in one static binary
`go-chi/chi`	HTTP	Routing for the daemon API
`gorilla/websocket`	Streaming	Live stdout and stderr from a running job
`quic-go`	Transport	Encrypted, multiplexed peer links over the punched UDP socket
`crypto/tls`, `ed25519`	Security	Pinned TLS on the daemon port; node identity and signing keys
Nix (pinned)	Environments	Content-addressed closures; the store path is the cache key; closure signing
`crun`	Sandboxing	OCI container per job, including GPU device passthrough
cgroup v2, `systemd-run`	Resources	Memory ceilings for the daemon and for each job
zstd (`klauspost/compress`)	Compression	Workspace and results archives
`modernc.org/sqlite`	Storage	Node store, in pure Go so the binary stays static
`gopsutil`	Metrics	CPU, memory and process-tree measurements
Cobra, Viper	CLI	Command structure and configuration
Bubble Tea, Lip Gloss	TUI	Live dashboard of nodes and jobs
NATS (embedded) [8]	Messaging	Optional alternate transport for lease messages
`zerolog`	Logging	Structured, levelled logging
Python	Interception	The shim (3,822 lines), embedded in the binary and injected per job
cloudpickle	Interception	Shipping lambdas and closures by value; added to every environment

1.5 Assumptions and Constraints

- Every participating machine runs the same binary. `pipedpeer setup` installs Nix and `crun` when they are missing, and can do so without root by using a private Nix store.

- Workers run Linux, since `crun`, cgroups and the Nix store are Linux-specific. The submitting side also builds for macOS as a client.
- Machines in a cluster share one secret. The secret is optional: a daemon with none configured accepts every request that reaches it (Section 6.3).
- Joining across networks needs one machine with a public address to act as introducer. It carries no job data.
- Two machines between which no direct path exists cannot work together; the peer is reported and routed around rather than relayed.
- The submitting process must remain alive for the duration of a job: it holds the lease and receives the output stream.

Chapter 2

Literature Survey

This chapter surveys work across the areas PipedPeer draws on: lightweight sandboxing and environment portability at the edge, messaging and actor-style fault tolerance, failure-transparent programming models, portability in the Python ecosystem, scheduling, and collaborative task offloading. Each entry closes with the bearing it has on the design decisions taken in Chapter 4.

Paper 1: Web Application-Based WebAssembly Container Platform for Extreme Edge Computing

Authors: S. Sekigawa, C. Sasaki, A. Tagami **In:** Proc. IEEE GLOBECOM, 2023 [9]

Summary: A platform that deploys WebAssembly modules as isolated compute units on edge nodes, sandboxing and executing user-submitted binaries without a container daemon such as Docker. Evaluation on constrained hardware shows acceptable latency with strong isolation.

Bearing on this work: The paper's argument that a lightweight sandbox can replace a full container runtime is the one PipedPeer follows, but the conclusion differs. PipedPeer must run arbitrary CPython with native extensions, which Wasm cannot host, so it takes an OCI runtime (`crun`) directly rather than a bytecode sandbox. Section 4.8 measures what that choice costs: roughly 20 ms per job.

Paper 2: WebAssembly for Edge Computing: Potential and Challenges

Authors: M. N. Hoque, K. A. Harras **In:** IEEE Commun. Standards Mag., vol. 6, no. 4, 2022 [10]

Summary: A survey weighing Wasm's near-native speed and memory sandboxing against containers and VMs, identifying WASI immaturity, limited file-system and networking APIs, and absent GPU support as the blocking gaps.

Bearing on this work: These gaps are precisely why PipedPeer ships a complete native environment as a Nix closure rather than compiling user code to a portable bytecode. A Nix closure carries native dependencies and GPU libraries intact, which is a hard

requirement for the machine-learning workloads in Section 4.14.

Paper 3: A Cross-Architecture Evaluation of WebAssembly in the Cloud-Edge Continuum

Authors: S. Kakati, M. Brorsson **In:** Proc. IEEE/ACM CCGrid, 2024 [11]

Summary: Benchmarks Wasm runtimes across x86_64, ARM64 and RISC-V, reporting 70–90% of native performance and sub-10 ms startup, with I/O-bound workloads suffering disproportionately from WASI syscall translation.

Bearing on this work: PipedPeer targets heterogeneous architectures too, but resolves them at build time rather than at run time: the flake generator emits an architecture-specific closure, and architecture is a *hard filter* during placement (Section 4.4) rather than something the runtime papers over. Full native performance is retained.

Paper 4: Design of Elixir-Based Edge Server for Responsive IoT Applications

Authors: Y. Li, S. Fujita **In:** Proc. IEEE CANDARW, 2023 [12]

Summary: An IoT edge server built on Elixir and OTP, where supervisor trees restart failed processes automatically and the system stays available through individual crashes.

Bearing on this work: Supervision is the model PipedPeer's retry loop follows: a failed placement is retried on the next-best node with exponential backoff, and the work is never dropped. The difference is where recovery state lives. PipedPeer has no supervisor process; recovery falls out of lease expiry (Section 4.5), so there is nothing whose own failure needs supervising.

Paper 5: A Synergistic Elixir-EDA-MQTT Framework for Smart Transportation

Authors: Y. Zhou, X. Chen **In:** Computers, vol. 16, no. 3, 2024 [13]

Summary: Combines Elixir concurrency, event-driven architecture and MQTT publish-subscribe, showing lower overhead than REST for high-frequency telemetry.

Bearing on this work: An earlier iteration of PipedPeer used publish-subscribe messaging (NATS) as its dispatch transport for exactly the reasons this paper gives. Experience moved the system the other way: bulk transfer of closures, workspaces and results needs streaming and backpressure, so dispatch is now plain HTTP with a WebSocket for output,

and messaging is retained only as an optional transport for small lease messages. The paper documents the case for the road not taken.

Paper 6: WebAssembly with wasi-nn for Edge Machine Learning Inference

Authors: J. Bachmeier, V. Yussupov, J. Henß, H. Koziolek **In:** Proc. ECSA, 2025 [14]

Summary: Evaluates Wasm with wasi-nn for edge inference, finding good portability for CPU inference and immaturity for GPU acceleration.

Bearing on this work: Machine-learning jobs are a primary use case, which is why `torch` and `tensorflow` carry the largest entries in PipedPeer's memory-estimation table and why GPU passthrough is implemented at the OCI level. This paper's negative result on GPU support under Wasm supports the decision to ship a native CUDA-capable environment instead.

Paper 7: Case Study of WebAssembly Runtimes for AI Applications on the Edge

Authors: S. E. Khelifa, M. Bagaa, A. O. Messaoud, A. Ksentini **In:** 2024 [15]

Summary: Compares four Wasm runtimes for AI inference on edge devices, finding memory consumption to be the dominant constraint.

Bearing on this work: Memory as the binding constraint is the assumption behind PipedPeer's tiered memory estimator and behind admission control: a node refuses a job it cannot hold, the pool executor splits a request that would exceed 40% of available memory into sequential chunks instead of failing, and on a node with a usable cgroup hierarchy every job runs under its own memory ceiling inside a daemon that is itself capped (Section 4.8).

Paper 8: Programming Models for Failure-Transparent Distributed Systems

Authors: A. Bergström **In:** M.S. thesis, KTH Royal Institute of Technology, 2025 [16]

Summary: Surveys abstractions that hide distributed failure from application code — actors, reactive streams, supervision — handling crash and omission failures by retry, replication and migration without exposing failure semantics upward.

Bearing on this work: This is the closest match to PipedPeer's central invariant. The user's script contains no failure handling at all, so every remote path must degrade to a

working local one. Section 4.11 sets out the five layers that enforce this, and Section 5.6 shows a worker being killed mid-computation without affecting the result.

Paper 9: Python Array API Standard

Authors: A. Meurer et al. **In:** Proc. SciPy, 2023 [17]

Summary: A standardised array API letting the same code run on NumPy, CuPy or JAX without modification, so scripts become portable across computational backends.

Bearing on this work: The goal is the same as PipedPeer's — portability without code change — but the mechanism is the mirror image. The Array API asks libraries to converge on one interface that users then adopt. PipedPeer changes nothing the user writes and instead substitutes the implementations behind the interfaces they already use. The comparison is drawn out in Section 5.11.

Paper 10: iDDS: Intelligent Distributed Dispatch and Scheduling

Authors: W. Guan et al. **In:** arXiv:2510.02930, 2025 [18]

Summary: Workflow orchestration that assigns tasks to heterogeneous resources from real-time CPU, memory, data-locality and dependency constraints, driven by a task DAG.

Bearing on this work: The scoring-based assignment is directly comparable to the placement function in Section 4.4. Two differences are worth stating: PipedPeer has no DAG, because it schedules whole scripts rather than task graphs, and it has no scheduler process at all, the scoring being performed in the submitting client. Data locality, which iDDS treats as first-class, is absent from PipedPeer's placement and is noted as future work.

Paper 11: Task Offloading in Multi-Hop Relay-Aided Multi-Access Edge Computing

Authors: Y. Deng, Z. Chen, X. Chen, Y. Fang **In:** IEEE Trans. Veh. Technol., vol. 72, no. 1, 2023 [19]

Summary: Formulates joint task assignment and relay selection to minimise completion time under transmission and energy constraints.

Bearing on this work: The offload-or-compute-locally trade-off is exactly what PipedPeer's cost model decides, and this paper's formulation of it as a comparison between transmission time and local computation time is the same shape as the inequality in Section 4.10. PipedPeer measures link bandwidth at run time rather than assuming a

channel model, and for parallel maps it goes further, fitting each peer's set-up time and throughput from the work it has actually returned.

Paper 12: Multi-Agent Deep Reinforcement Learning Based Dynamic Task Offloading

Authors: H. He, X. Yang, X. Mi, H. Shen, X. Liao **In:** Sensors, vol. 24, no. 16, 2024 [20]

Summary: Each device learns an offloading policy from local observations without a central coordinator, beating centralised heuristics on the energy-delay trade-off.

Bearing on this work: This establishes the ceiling PipedPeer's deterministic rule-based model gives up. The trade is deliberate: a learned policy needs training data and a training phase, which is incompatible with a tool whose value proposition is that a new machine joins by running one binary. It is a natural direction once a deployment produces enough history.

Paper 13: Joint Optimization of Multi-User Partial Offloading and Resource Allocation

Authors: D. Yong, R. Liu, X. Jia, Y. Gu **In:** Sensors, vol. 23, no. 5, 2023 [21]

Summary: Optimises the split between locally executed and offloaded portions of a task, finding partial offloading consistently better than all-or-nothing.

Bearing on this work: This paper describes what PipedPeer now does. Interception splits a single script's work across nodes at the granularity of `Pool` chunks, hash-shuffle buckets and matrix row blocks, while the remainder runs locally — partial offloading in the sense used here. The split is chosen as this paper recommends, by minimising completion time: each peer's share comes from a makespan solver over measured rates (Section 4.9). The racing scheme in Section 4.11 goes further by keeping the local share speculative, so an unresponsive remote share costs nothing.

Paper 14: Energy-Efficient Collaborative Task Offloading in MEC

Authors: L. Chen et al. **In:** Ad Hoc Networks, vol. 169, 2025 [22]

Summary: Devices cooperate by relaying tasks toward edge servers, with a learned policy cutting total energy against non-cooperative baselines.

Bearing on this work: The symmetric model, where every device is both client and server, is PipedPeer's model too: one binary, and any node may submit or accept. Piped-

Peer also forwards a parallel-map request that a node cannot hold to a healthier peer rather than refusing it.

Paper 15: Blockchain Enabled Task Offloading in Digital Twin Vehicular Edge Networks

Authors: C. Li, Q. Chen, M. Chen, Z. Su, Y. Ding, D. Lan, A. Taherkordi **In:** J. Cloud Comput., vol. 12, no. 120, 2023 [23]

Summary: Smart contracts enforce service agreements between task submitters and edge workers, preventing free-riding in an untrusted network.

Bearing on this work: PipedPeer now authenticates who may use a node, with a cluster-wide shared secret, encrypted links and signed closures (Section 4.3). It does not address what this paper addresses, which is whether a participant that *is* admitted behaves well: every holder of the secret can run arbitrary code on every node, and nothing accounts for or rewards contribution. The paper describes the class of mechanism that would be needed for a cluster of mutually untrusting owners, and its commit-and-verify structure resembles the existing three-phase lease.

Paper 16: Congestion-Aware Distributed Task Offloading Using Graph Neural Networks

Authors: Z. Zhao, J. Perazzone, G. Verma, S. Segarra **In:** arXiv:2312.02471, 2023 [24]

Summary: Trains a graph neural network to produce distributed offloading policies from local neighbourhood observations, outperforming greedy baselines under congestion.

Bearing on this work: PipedPeer scores each node independently on scalar metrics and is therefore blind to topology: two nodes with identical load score identically even if one is a slow hop away. Link cost is measured, but only where work is divided — in the cost model's bandwidth probe and in the per-peer set-up time fitted for parallel maps — and not when a whole job is placed. Feeding a graph-structured view into placement is the most concrete extension this survey suggests.

Standards Underlying Cross-Network Operation

Joining machines on different networks (Section 4.2) rests on established standards rather than on new mechanisms, and the design follows them closely. Address discovery uses the reflexive-address idea of STUN [27], and the diagnostic `net-check` command classifies a router's mapping and filtering behaviour in the terms of RFC 5780 [28]. Port mapping

requests follow NAT-PMP [29], its successor PCP [30], and UPnP Internet Gateway Device [31]. Simultaneous UDP probing from the socket that will carry the connection is the hole-punching technique analysed by Ford, Srisuresh and Kegel [32], whose central observation — that a router's mapping belongs to a socket, so the probe and the connection must share one — shapes PipedPeer's shared-socket design. Links are QUIC [25] secured by TLS 1.3 [26], node identities are Ed25519 keys [33], and archives are compressed with Zstandard [34].

Chapter 3

Analysis

3.1 System Architecture

PipedPeer follows a **daemon, not CLI** architecture. Every machine runs one long-lived daemon that handles discovery, admission, execution and result delivery. The `pipedpeer` command is a short-lived client.

One consequence is worth stating early because it distinguishes the system from every framework in Chapter 2: **the coordinator is a library linked into the client, not a service**. There is no scheduler process, no head node and no master anywhere. A machine acts as submitter and worker simultaneously, and the same binary does both. A new participant on the same network joins by running that binary; one on another network runs `pipedpeer join` with the address of an **introducer**. The introducer is the one asymmetric role in the system, and it is optional: it is an address book that tells machines where to find each other, and it carries no job data.

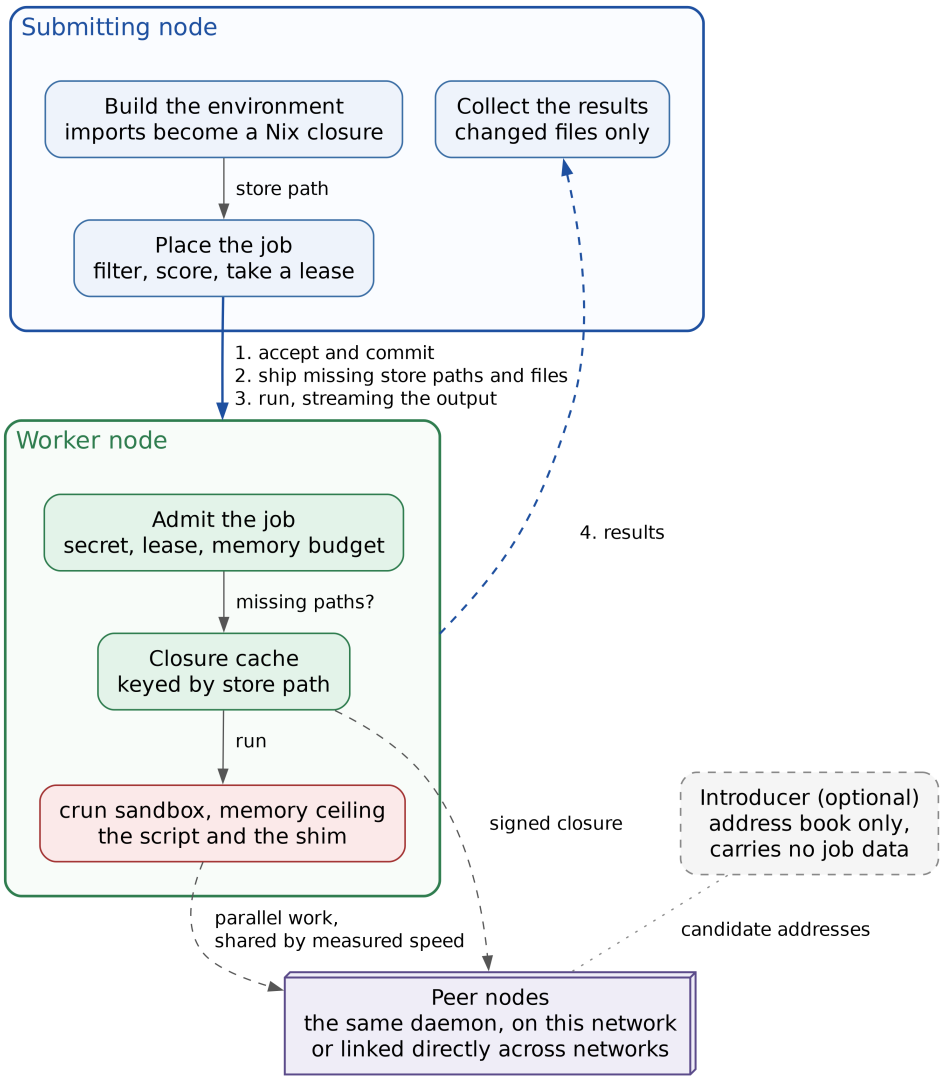


Figure 3.1: PipedPeer system design. The submitting side builds the environment and places the job; the worker admits it, caches the closure and runs it sandboxed under a memory ceiling. Both roles are the same binary. Peers on other networks are reached over direct QUIC links that an introducer helps establish but never carries.

The principal subsystems are listed in Table 3.1. All are packages under `src/internal/`; there are 37 of them, the remainder being supporting code (process management, set-up, XDG paths, logging and the like).

Table 3.1: Subsystems and their responsibilities

Package	Responsibility
`coordinator`	Filters, scores and selects a node; takes the lease; retries on failure
`daemonapi`	HTTP and WebSocket surface (22 routes), lease manager, sandbox, pool executor, gradient reduction, closure caches
`schedule`	Makespan-optimal division of a chunk across measured devices
`ddpplace`	Choosing the ranks of a training ring by measured throughput
`discovery`	UDP multicast and broadcast probes, TCP health sweep fallback
`internet`	Peer manager across networks: publish candidates, punch, link, forward
`direct`	Shared UDP socket, candidate race, birthday-collision fallback, QUIC links
`rendezvous`	The introducer's address book and punch-request forwarding
`portmap`	Router port mapping by PCP, NAT-PMP and UPnP-IGD
`authtoken`, `tlsid`	Shared-secret middleware; self-signed certificate, pinning, TLS and HTTP on one port
`identity`, `nixsign`	Node UUID and ed25519 key; signing closures and the trusted-key list
`narpack`	Signed closure archives and least-recently-used cache pruning
`nixstore`	System or private (rootless) Nix store
`cgroups`	Memory ceilings for the daemon and for each job; the daemon's budget
`nodestore`	SQLite record of known peers and their last observed capacity
`nixgen`	Flake generation, wheel pins, and the interception shim as an embedded asset
`pythondeps`	Import scanning and import-to-package resolution
`resourceest`	Tiered memory estimate
`jobhistory`, `heartbeat`, `gpu`	Job records; capability and load reporting; vendor GPU detection
`registry`	Optional standalone directory

3.2 Data Flow Diagram

Level 0 — Context Diagram

At the highest level the system has four external entities: the **user**, who submits a script and receives output and result files; **peer nodes**, which supply capacity and return computed results; an optional **introducer**, which exchanges candidate addresses so that peers on other networks can reach one another; and an optional **registry**, a directory of

nodes.

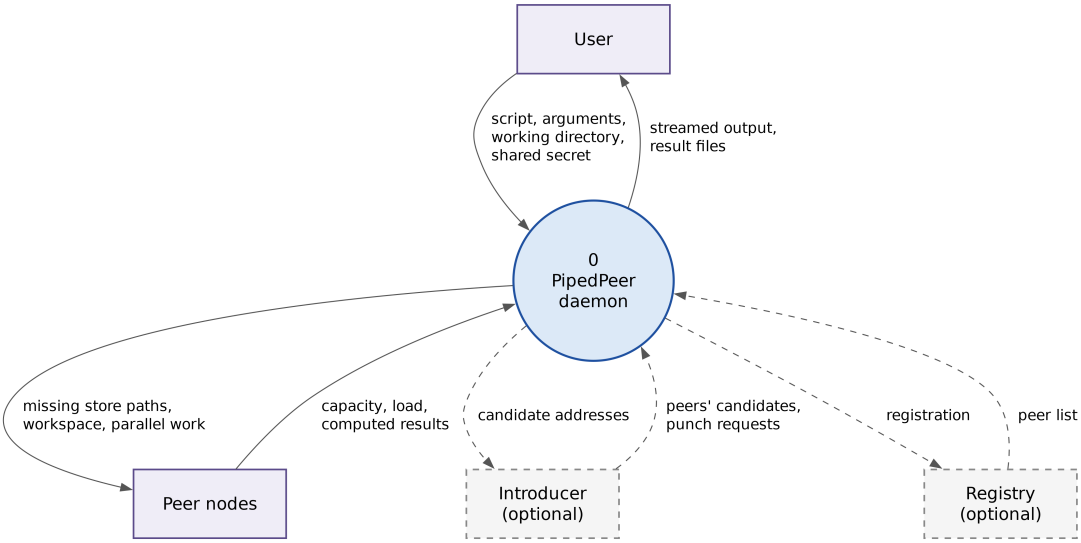


Figure 3.2: Level 0 DFD — context diagram. The introducer and the registry are dashed because the system works without them.

Level 1 — System Decomposition

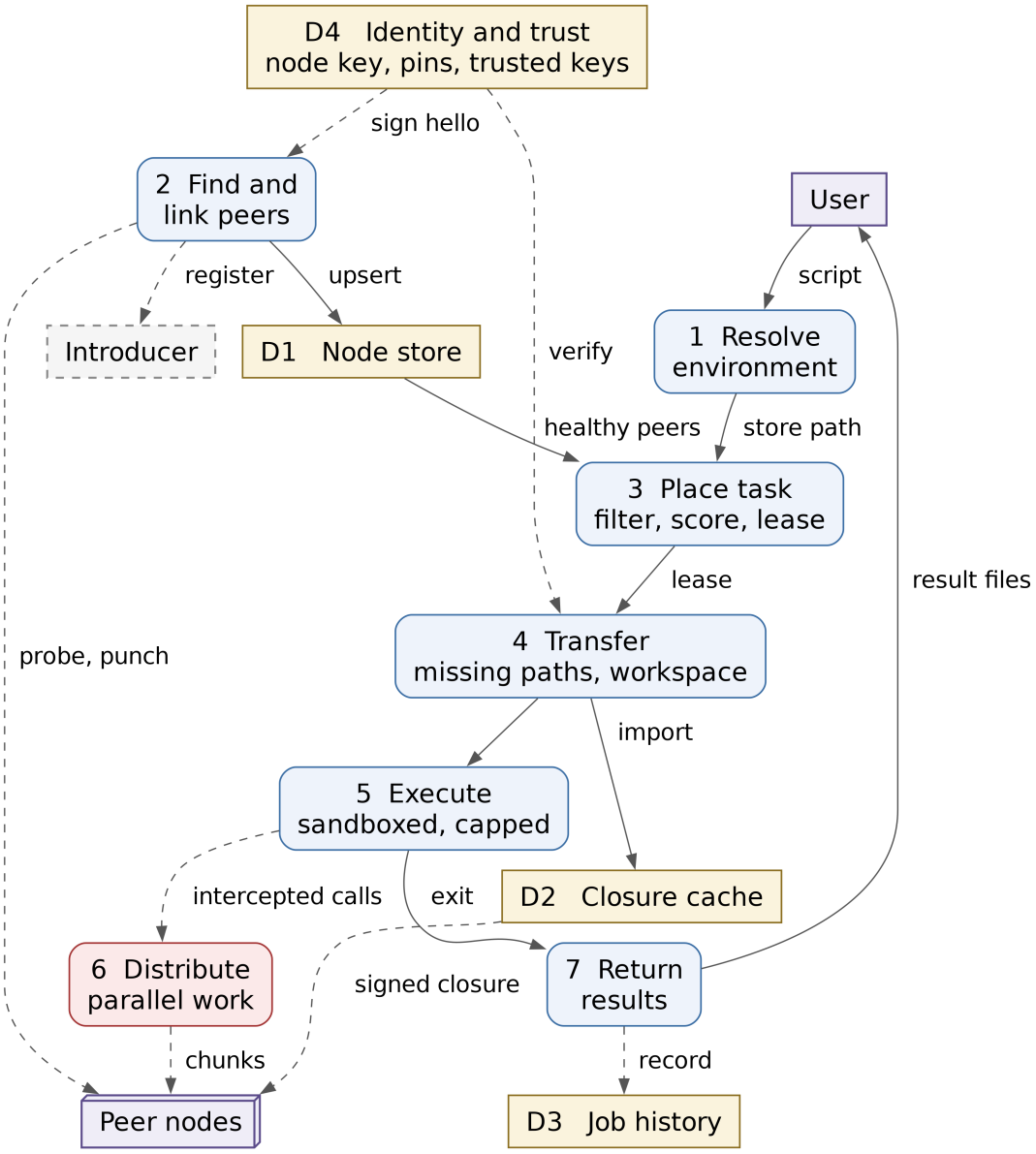


Figure 3.3: Level 1 DFD — seven processes over four data stores.

- **1 Resolve environment.** Scan the script's imports, generate a flake, build the closure.
- **2 Find and link peers.** Probe the subnet; across networks, publish candidates to the introducer, punch, and register each direct link as a node.
- **3 Place task.** Apply hard filters, score the survivors, reserve a lease on the best.
- **4 Transfer.** Upload the workspace always, and of the closure only the store paths the worker lacks; signed between daemons.

- **5 Execute.** Run inside a `crun` sandbox under a memory ceiling, streaming output back line by line.
- **6 Distribute parallel work.** Share chunks among peers by measured throughput, subject to the cost model.
- **7 Return results.** Send back changed files and a deletion manifest.

The four stores are the node store, the closure cache, job history, and the identity and trust store (node key, pinned certificates, trusted Nix keys).

3.3 Use Case Diagram

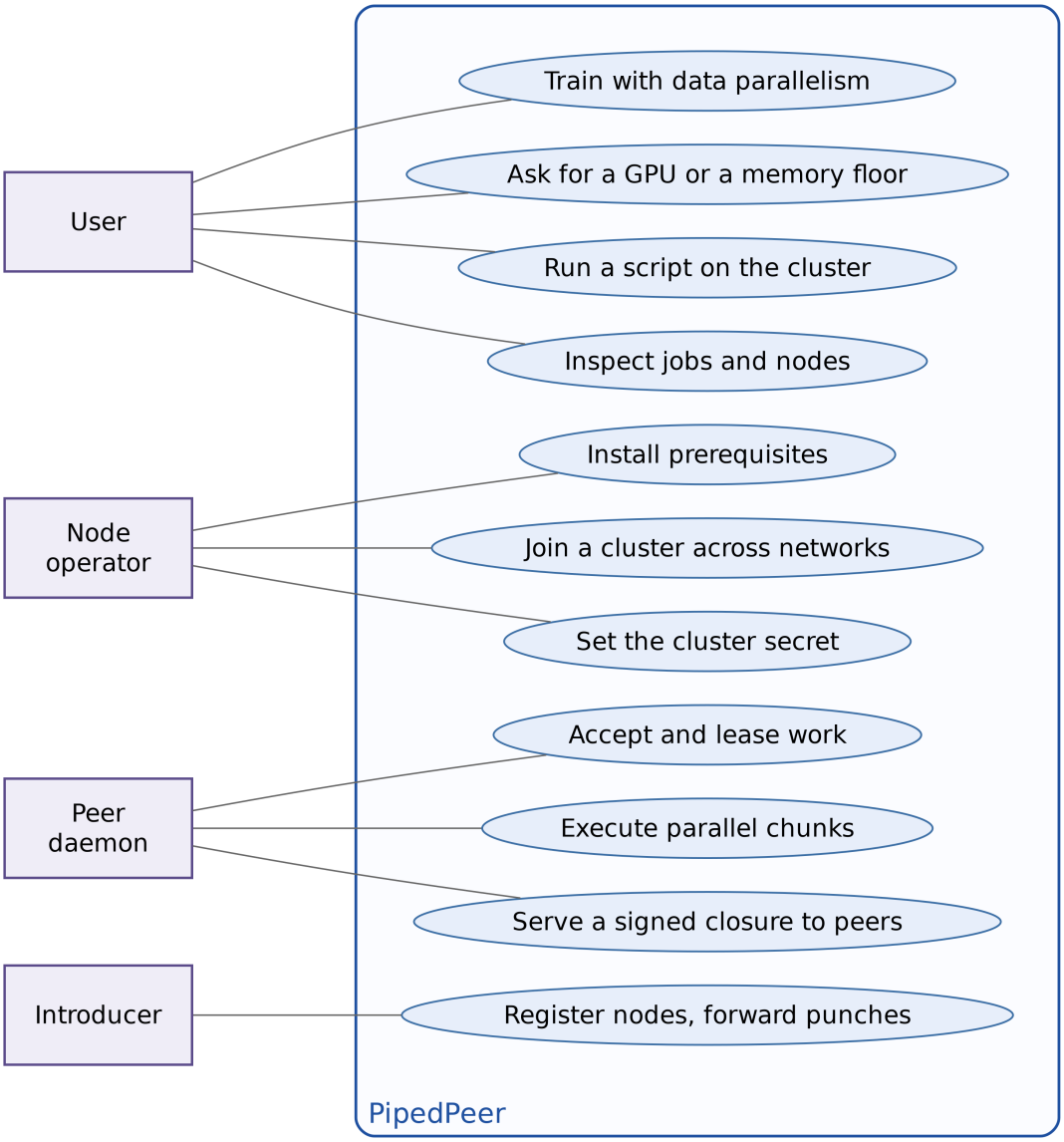


Figure 3.4: Use cases. A peer daemon is an actor in its own right, because every node serves closures and executes chunks for others; the introducer registers nodes and forwards punch requests.

3.4 Sequence Diagram

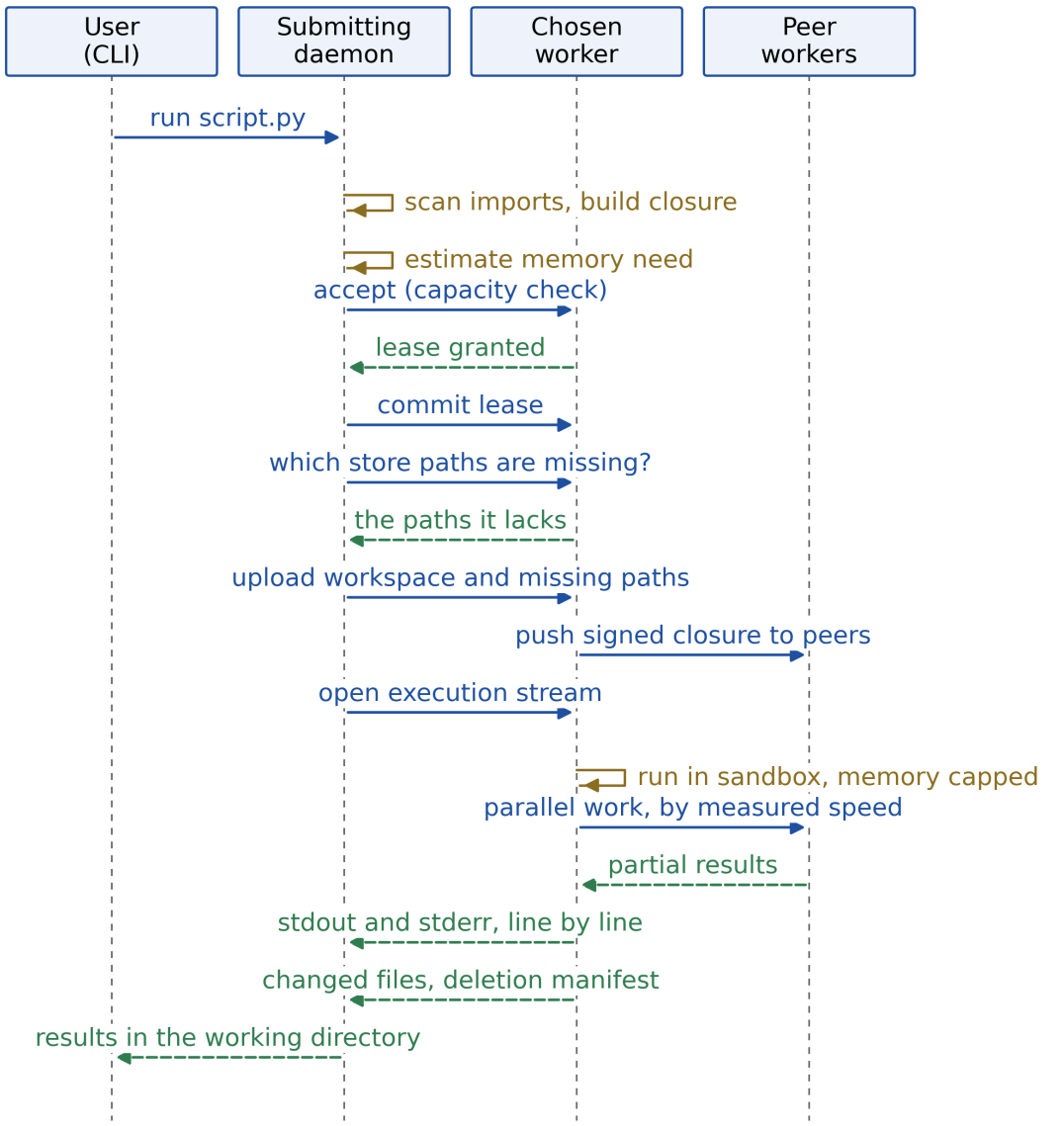


Figure 3.5: End-to-end execution of one job.

1. The user runs `pipedpeer run script.py`.
2. The client scans imports and builds a Nix closure locally, then estimates the job's memory requirement.
3. It asks its own daemon for the list of healthy peers, which includes peers on other networks, reached through local forwarders onto direct links.
4. The coordinator filters and scores them and sends `accept` to the best candidate, which checks its capacity and reserves it.

5. On acceptance the client sends `commit`, which turns the reservation into a running lease.
6. It asks whether the worker can already run the closure. If not, it asks which store paths are missing and sends only those, falling back to the whole closure; then it uploads the workspace.
7. The worker pushes the closure to peers that lack it, so they become eligible to receive spilled work; peers that accept the signed format receive a signed archive.
8. Execution proceeds inside a sandbox under a per-job memory ceiling, with stdout and stderr streamed back a line at a time.
9. If the script uses a parallel primitive, the shim distributes it subject to the cost model.
10. On exit, changed files and a deletion manifest return as a zstd archive and are extracted into the user's directory.

Every request in this sequence carries the cluster's shared secret when one is configured, and travels over pinned TLS on a local network or inside a QUIC link across networks.

Chapter 4

Design and Methodology

4.1 Node Discovery on a Local Network

Three sources feed one table, and none blocks the others.

UDP probes. The daemon opens two sockets on port `38099`, one joined to multicast group `239.255.0.99` and one bound for broadcast, both with `SO_REUSEADDR` and `SO_REUSEPORT`. The probe is a fixed magic string and the reply carries a JSON node descriptor. The multicast socket exists for a specific reason: a UDP broadcast to a shared port is delivered by the kernel to only one socket, so two daemons on the same host would never see each other. Multicast with port reuse reaches all of them.

This mechanism was described as mDNS in earlier documentation of this project, and the dashboard labelled it so. It is not: there is no DNS-SD, no service record and no conflict resolution. It is a bare request-response probe, and the dashboard now labels it "broadcast".

TCP fallback. Access points with client isolation drop broadcast traffic without error. When the UDP probe returns nothing, the daemon sweeps the first 254 addresses of each local subnet with concurrent health probes, 64 at a time with a 700 ms timeout each, and stops issuing probes once any daemon answers.

Freshness. A peer poller runs every 10 s with a 3 s timeout, and a single failed poll marks a node unreachable at once. Two further thresholds apply: a row not refreshed for **45 seconds** — for example while the local daemon itself was down — is not reported as healthy, and after **5 minutes** an auto-discovered row is deleted outright. Manually added peers, which include the peers linked across networks in Section 4.2, are never removed by the second rule.

4.2 Joining Across Networks

A broadcast cannot leave the local network, and a machine behind another router will not accept an inbound connection. Figure 4.1 shows how two such machines come to share a direct, encrypted link without either of them opening a port by hand and without any machine relaying their traffic.

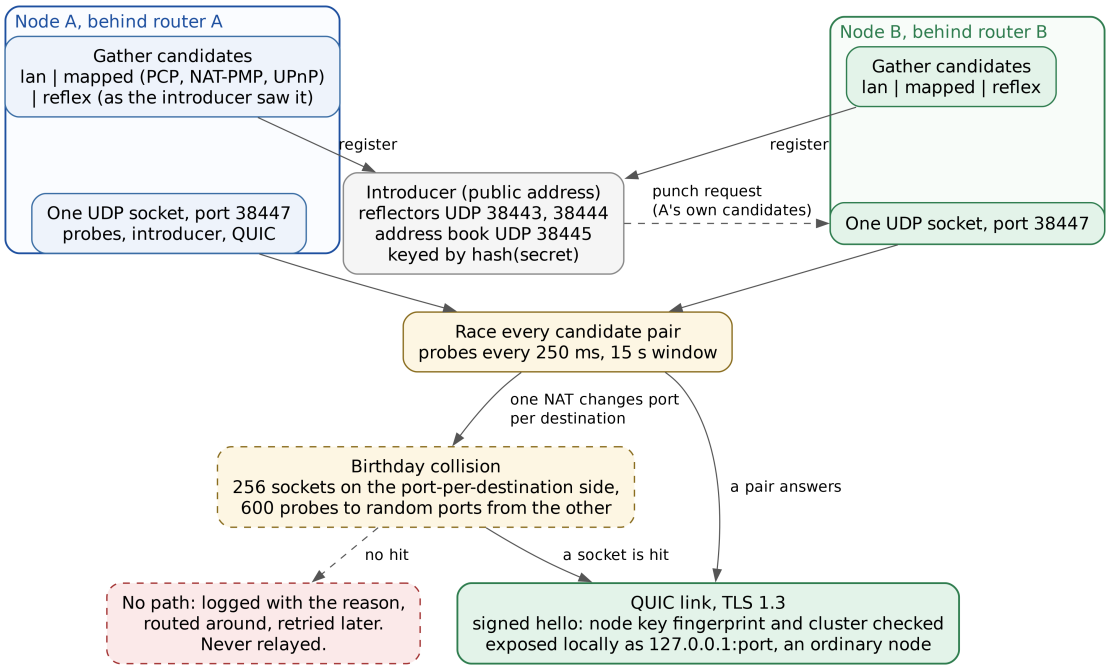


Figure 4.1: Joining across networks. The introducer exchanges candidate addresses and forwards one punch request; the two nodes then race their candidate pairs over a single UDP socket, fall back to a birthday collision when one NAT allocates a port per destination, and carry everything over a mutually authenticated QUIC link exposed locally as an ordinary node.

One command. `pipedpeer join <introducer>` normalises the address (default port 38445), saves it so that a later `start` rejoins, restarts the daemon in internet mode, and waits up to 90 s for peers before printing the node table. `pipedpeer start --rendezvous` does the same without waiting.

The introducer. `pipedpeer rendezvous` runs two reflectors (UDP 38443 and 38444) that report the address a probe arrived from, in the manner of STUN [27], and an address book on UDP 38445. Nodes register their candidate addresses under a cluster identifier derived from the shared secret by hashing, so the introducer never holds the secret itself. A `connect` request causes it to forward a single punch datagram to the target, carrying only the candidates the caller itself registered; requests are rate-limited per source to four per second. It carries no job data. Measured over a whole verification run, the introducer's host sent 0.2 MB while about thirty megabytes crossed between the peers.

Candidates. Each node offers three kinds of address: its LAN addresses; a port its router agreed to forward, requested by PCP [30], NAT-PMP [29] or UPnP-IGD [31] for two hours and renewed at half the lifetime actually granted; and the reflexive address the introducer observed. A router mapping is withheld when its external IP address differs from the reflexive one (once the reflexive address is known; on the very first poll it is not yet, and the mapping is published unchecked). Behind a second layer of NAT, such as a

phone tether, the router that answers is not the one facing the internet, and publishing its mapping would advertise an address nobody can reach.

One socket. Punch probes, introducer messages and QUIC share a single UDP socket on port 38447. This is the mechanism, not a convenience: a router's mapping belongs to the socket that created it [32], so the connection must arrive on the socket the probes left from. Incoming datagrams are separated by content — probes by four magic bytes, introducer messages by a leading brace, everything else handed to QUIC.

The race. Both sides probe every pairing of their candidates every 250 ms for a fifteen-second window, rather than trying candidates one at a time, which would add a timeout for every address that cannot work. When one side's NAT allocates a fresh external port for every destination there is no address to publish, and an ordinary punch fails: on a phone tether the project's own laptop was seen as port 38471 by one reflector and 27486 by the next, and a punch moved 400 packets each way without any arriving. For that pair the **birthday collision** runs: the side with the port-per-destination NAT opens 256 sockets toward the peer, the other sends 600 probes to random ports, and the socket that is hit is the one kept, since its mapping is the reachable one.

The link. The winning path carries QUIC [25] with TLS 1.3. Inside it each side sends a hello signed with its Ed25519 node key, and each checks that the key's fingerprint is the node it meant to reach and that the cluster identifier matches. TLS on this link skips certificate checks, so a node's identity rests on the hello. The hello therefore signs the session's exported keying material [36] and proves the cluster secret with an HMAC over the same value, so it is valid for one session and one cluster only; an earlier hello, which signed only the cluster and a nonce the signer chose, could be replayed by a peer to a third node to pass as its sender. A node on an older build is refused and told why. The link is then exposed to the rest of the daemon as a TCP forwarder on a loopback port, registered as an ordinary node; each local connection becomes one QUIC stream to the peer's daemon. Placement, closure transfer and gradient exchange therefore work over the link unchanged. `pipedpeer nodes` reports how each peer is reached: `self`, `lan`, `mapped`, `reflex`, `punched` for a collision, or `punched-in` when the peer dialled.

No relay. A relay was built and then removed from the data path, because a relay carries every byte of every job and becomes the cluster's bottleneck and its cost. A peer with no direct path is logged with the reason, never offered to the scheduler, and retried after a back-off; `pipedpeer nodes` lists it under the table with that reason. A live link does not depend on the introducer: with the introducer suspended, a peer was reached in 0.03 s.

What a NAT lab found. Until a container lab with real translators existed (Section 4.14), punching was exercised only at live demonstrations. The lab showed peers behind NAT rarely linking, for four reasons, all since fixed. The socket a birthday col-

lision found listened under the search's context, which ended on adoption, so the peer's dial met a closed listener; it now accepts dials for 30 s and stays open while any link it accepted lives. The same socket kept the search's read deadline, so after adoption every read failed and its links died about 90 s later; the deadline is now cleared. A node registered before it knew its public address, so a woken peer punched at a private address on another network; it now registers again as soon as it learns that address, before asking anyone to punch. And the race returned an IPv4-mapped address that never matched a candidate, so an ordinary punch was labelled a collision.

4.3 Security and Identity

The design documented previously assumed a trusted local network and had no authentication of any kind. The current design has three layers (Figure 4.2). Each is optional in the sense that a node without it still runs, which keeps a single-machine or classroom setup to one command, and each is described here together with what it does not cover.

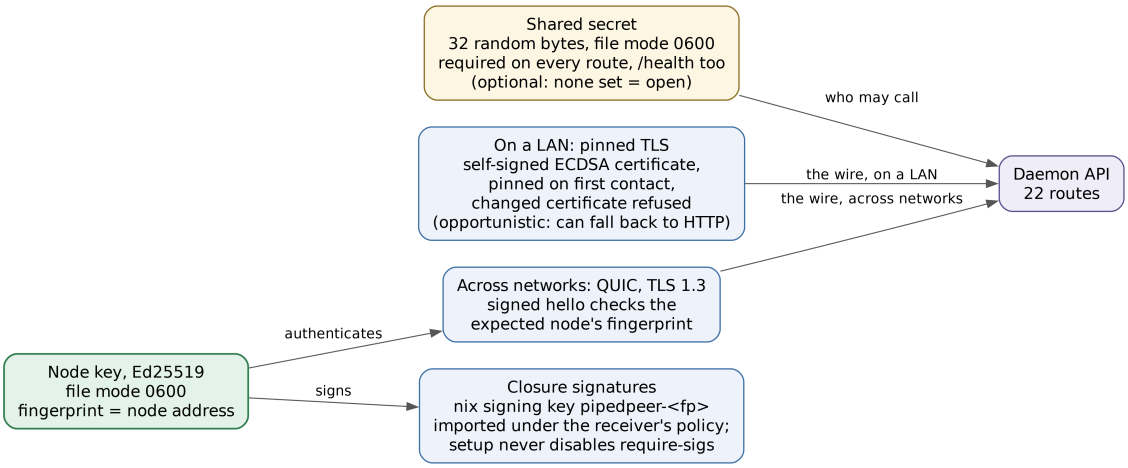


Figure 4.2: The three layers of trust: a shared secret on every API route, pinned TLS (or QUIC across networks) on the wire, and the node key that both authenticates links and signs closures.

The shared secret. `pipedpeer auth set` stores 32 random bytes, or a given secret, readable only by the user. Every route on the daemon, `/health` included, then requires it, compared in constant time; outbound requests carry it automatically. Jobs receive it in their environment so that the shim can reach its own daemon, which means a job's code can read it. With no secret configured the daemon accepts every request, which is the earlier behaviour. The secret is attached only to a host that has proved it holds it: a remote host is first challenged and must return an HMAC over the challenge, whereas the LAN sweep once sent the secret in plain HTTP to every host on the subnet that answered on the port.

Pinned TLS. Each daemon creates a self-signed ECDSA certificate on first start. TLS and plain HTTP share the daemon port; the listener inspects the first byte of each connection and treats a TLS handshake record as TLS. Clients try TLS first and pin the peer's certificate on first contact; a changed certificate is refused, with the command that clears the pin. The scheme is trust on first use, and it is opportunistic: a handshake that fails for any reason other than a pin mismatch makes the client remember that peer as plain HTTP. The code records that an attacker able to break the handshake can therefore force plain text. Peers across networks are reached through loopback forwarders, so their traffic is encrypted by QUIC rather than by this layer.

Node keys and signed closures. Each node holds an Ed25519 key [33]. The same key authenticates QUIC links and signs closures as a Nix signing key. Nodes publish the public half in their heartbeat, each daemon keeps a list of the keys of the nodes it knows, and `setup` adds those keys to the system Nix configuration as additional trusted keys, with consent. It never disables Nix's signature requirement; an earlier version did, switching off signature checking for everything the machine installed so that one program could receive unsigned closures. A closure sent between daemons in the signed format is imported under the receiver's own signature policy. On a machine with signatures required, a closure signed by a trusted peer imported and ran, and the same closure was refused once the sender's key was removed from the list.

Getting peer keys to Nix. On a fresh cluster the first job never left its submitter: every worker refused the closure as lacking a trusted signature, because the worker knew the sender's key only in its own list and Nix trusts only what its configuration names. Three changes close this without relaxing the signature requirement. The import passes the cluster's keys to that one copy operation. An archive names its signer and key, which the receiver checks against the signer's published identity before trusting it for that import, so a job sent right after joining is not refused for want of a poll. And because a multi-user Nix daemon ignores keys passed by a user it does not trust, set-up installs, with the one privileged step it already needs, a root-owned copy of the binary and a rule letting the user run only that file; run under that name, the binary does nothing but add or remove keys of the cluster's fixed form, each in its own configuration include, and restarts the Nix daemon. The daemon calls it for a key system Nix does not yet trust, before importing, and retries a failure every 15 minutes. A refused import is reported to the submitter.

What remains uncovered. The secret is one per cluster, so there is no per-user or per-node authorisation, and every holder can run code on every node. The trusted-key list is built from every node in the node store, including nodes found by unauthenticated LAN discovery, so a key in that list is not bound to an authenticated identity. With the key helper installed, anyone holding the cluster secret can get signed paths trusted by system Nix on every machine with no human involved, as was already true after a manual trust

step. The older closure format, which a submitter still uses for its own upload, is imported without a signature check. The optional registry and LAN discovery are unauthenticated. Section 6.3 returns to these.

4.4 Placement and Scoring

Candidates are merged from the node store, live discovery, the optional registry and the local node, deduplicated by node identifier. Five hard filters then run in order: architecture must match the closure; a GPU must be present if one was required; and free memory, free cores and free VRAM must each satisfy the request. Every rejection records a reason, which surfaces in `pipedpeer job`.

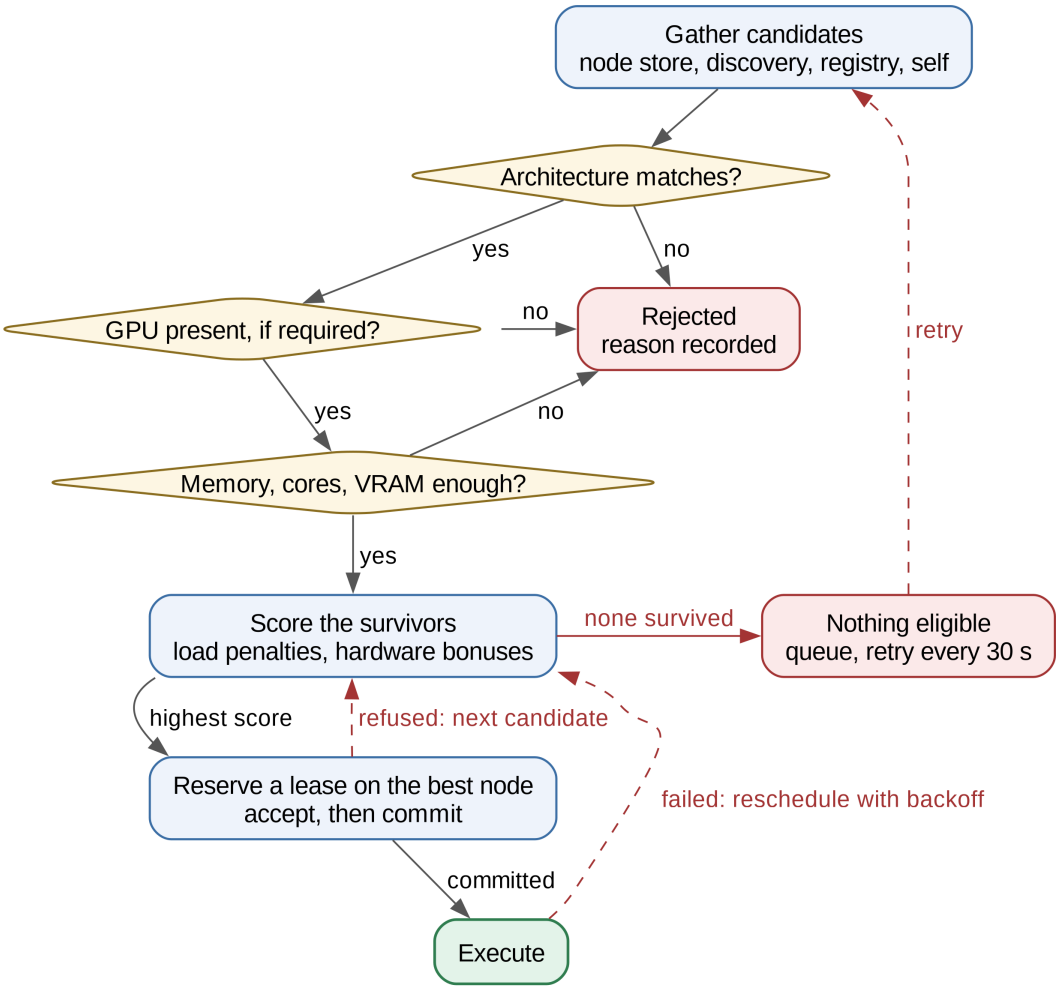


Figure 4.3: Placement: gather, filter, score, lease, retry. A task is never dropped; when nothing is eligible it is queued and retried.

Survivors are scored. Writing u_{cpu} and u_{mem} for CPU and memory utilisation in percent, n_{jobs} for the count of running jobs, and $\hat{c}, \hat{f}, \hat{m}, \hat{r}$ for the core count, clock speed, free

memory and free-core ratio each normalised onto $[0, 1]$ against a saturation point:

$$S = \left(1 - \frac{u_{\text{cpu}}}{200} - \frac{u_{\text{mem}}}{200} - 0.05\, n_{\text{jobs}} + 0.10(\hat{c} - \tfrac{1}{2}) + 0.06(\hat{f} - \tfrac{1}{2}) + 0.10(\hat{m} - \tfrac{1}{2}) + 0.10(\hat{r} - \tfrac{1}{2})\right) \cdot H \tag{4.1}$$

where H is the node's health score. Two properties matter more than the individual coefficients.

Load dominates hardware. The two load terms can subtract 1.0 between them, while the four hardware terms together move the score by at most 0.36. A fast but busy machine therefore loses to a slower idle one, which is the intended behaviour on a network of workstations that people are also using.

Missing telemetry is neutral. The normalisation returns 0.5, the midpoint, for a value that is absent or non-positive, so a node that does not report its clock speed is neither rewarded nor punished. This is what makes the function safe across heterogeneous machines whose telemetry is uneven.

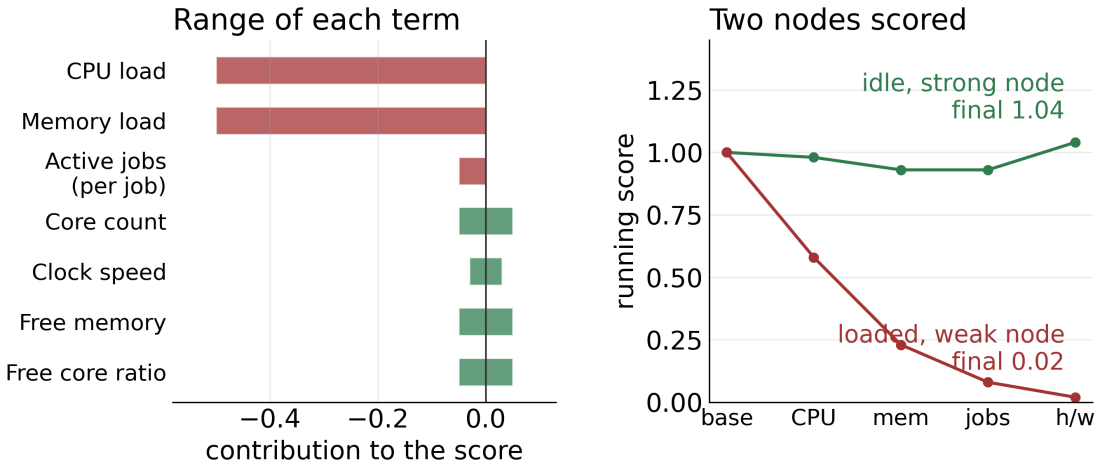


Figure 4.4: Left: the range each term can contribute. Right: the same function applied to an idle strong node and a loaded weak one.

When a GPU is requested, a second function contributes up to 0.8 more, drawn from total VRAM, device count, free VRAM on the best device and compute capability, less a utilisation penalty.

The scoring function is unchanged from the version documented in August 2026. What changed is the memory figure it reads: a node now advertises the memory it can actually *grant* — its own budget less the reservations of every daemon on the same machine — rather than the machine's free memory (Section 4.8).

4.5 The Lease Protocol

Placement is a three-phase handshake, and its state machine is where crash recovery lives.

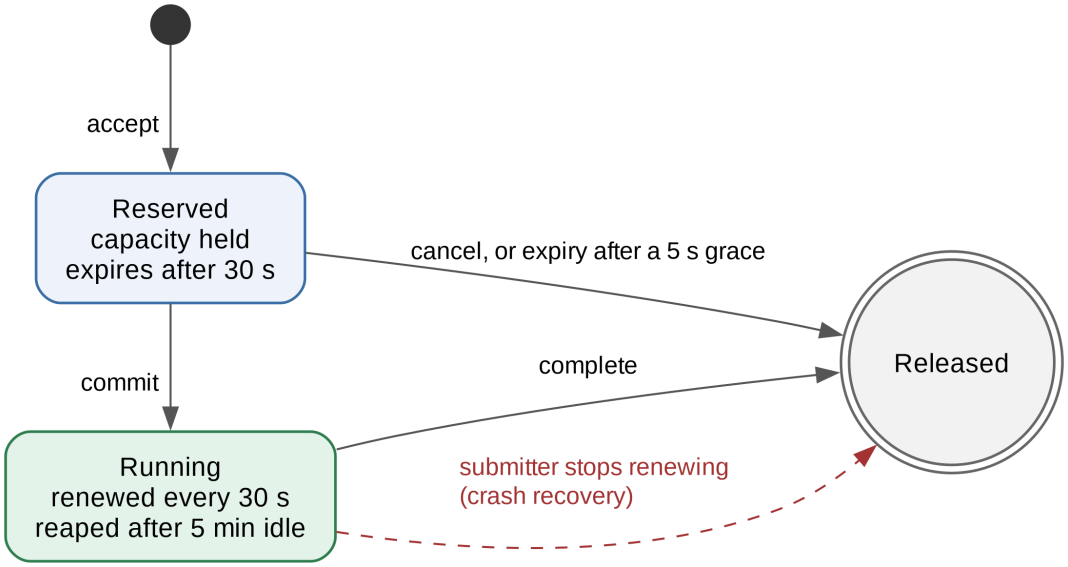


Figure 4.5: Lease states. Recovery is a consequence of expiry rather than a separate mechanism.

An `accept` reserves capacity for 30 seconds with a 5 second grace period; a `commit` promotes the reservation to running, renewed every 30 seconds and reaped after 5 minutes without renewal; `complete` releases it. A sweeper runs every 2 seconds.

Two details carry the correctness argument. First, the concurrency and memory checks and the lease insertion happen **under a single lock**, so two submitters racing for the last slot cannot both succeed (the cores check runs just before the lock is taken). Second, the concurrency limit counts reserved and running leases together, deliberately, so a burst of submitters cannot slip through the window between accept and commit. Admission checks cores, then the concurrency limit, then memory against the daemon's budget, then GPU devices.

Recovery then needs no additional machinery. A submitter that dies stops renewing and the sweeper reclaims its slot. A worker that dies stops answering health checks, is marked unreachable at the next failed poll, and its in-flight connection fails, which the coordinator treats as a failed attempt and reschedules elsewhere, with back-off doubling from 2 seconds to a 2 minute ceiling. Tasks are never dropped.

4.6 Reproducing the Environment

Import scanning. The client extracts imports with a hand-written, line-oriented parser rather than a full parse of the program. It strips comments, takes the module from `from X import …`, and splits `import a, b as c` into every module named; the earlier version used a single regular expression that kept only the first. Being line-oriented, it does not see an import split across continuation lines or performed through `__import__` or `importlib`, and it can pick up import-like text inside a multi-line string. Each name is then classified as a local file, a standard library module (from a list of 124), or an external dependency, and external names are resolved through a 40-entry mapping table that handles cases such as `sklearn` to `scikit-learn`.

Imports outside the table. Until October a third-party import absent from that table resolved to nothing and was silently omitted from the closure, so the job died on an import error on the node. Such an import is now looked up as the attribute of the same name in the pinned package set's Python packages, with one evaluation for all of them, and added when it exists. Only what is still missing is reported, before anything runs, with the flag that would add it; a failed evaluation says so and falls back to the warning. A `requirements.txt` beside the script is read as if it had been passed with `--pkg`. The scan also follows the script's own modules: a helper module's imports used to go unread, so a helper importing a plotting library left the environment without it. Each local module is now scanned against its own directory and then the script's, with a visited set so that import cycles end. A `uv.lock` file, which replaces detection entirely when present, and an explicit `--pkg` flag remain the escape hatches.

Common packages built ahead of time. There is no layered base environment; what environments share is Nix's own store paths, and a transfer sends only the paths a peer lacks (Section 4.7). A preparation step uses that: it builds one environment importing the usual scientific set (numpy, pandas, scikit-learn, scipy, matplotlib, seaborn, tqdm and CUDA torch) on every machine through the dry run of Section 4.13, so a script seen for the first time shares those paths and fetches only what it adds. A teacher-style script's environment took **54** s to build without that step and **4** s with it, measured on a container of the rehearsal lab (Section 5.10).

The keystone. Nix [5] names every build output by a hash of its inputs. The generated flake pins an *exact* nixpkgs revision, not a branch. The reasoning is the foundation of the entire caching layer. A branch resolves at build time, so two nodes building the same script on different days would produce different store paths; because the cache is keyed on the store path, the system would then re-ship a multi-hundred-megabyte archive for every job. Pinning an exact revision makes the store path a stable, content-derived name for an environment, identical across every machine and across time. Section 5.3 measures

what that buys.

Every generated environment also contains `cloudpickle`, so that parallel kernels with no source can be shipped by value (Section 4.9). Two deliberate exceptions to the package set exist, both restricted to x86_64 Linux: scikit-learn is pinned to a specific wheel because the package set lags the version targeted, and torch with CUDA is fetched as a pinned wheel set inside a *fixed-output derivation* with a recorded hash, because building it from source takes hours. The CUDA build is used only when a GPU is requested — which is the default when a script imports torch, unless `--gpu off` — and otherwise the package set's CPU build is used.

4.7 Closure Transfer and Cache Reuse

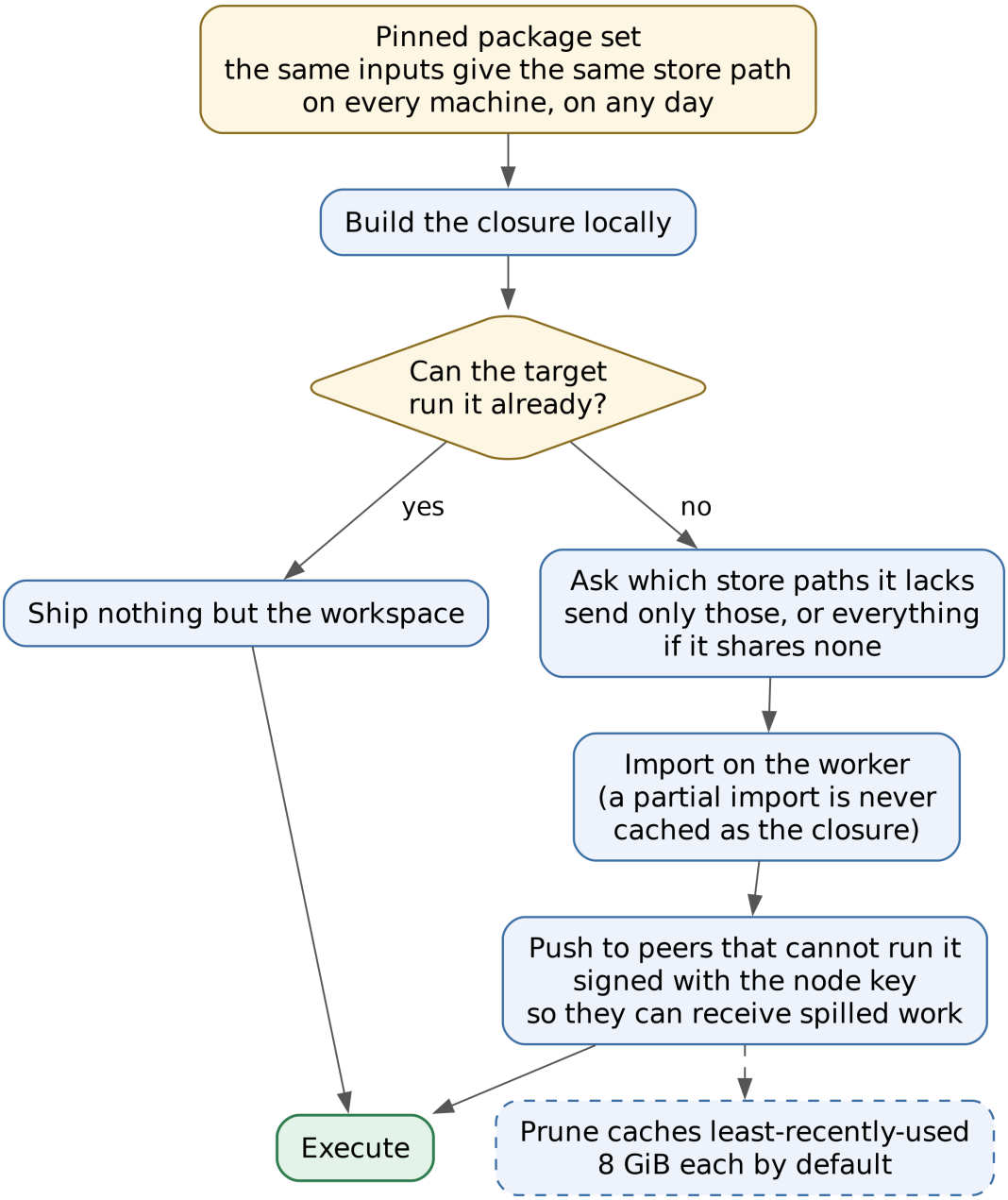


Figure 4.6: The store path is the cluster-wide cache key. Most submissions ship no environment at all, and a related environment ships only the store paths the receiver lacks.

Before uploading, the client asks the target whether it can already run the closure. Only on a negative answer does anything cross the wire, and then, where possible, only part of it.

There are in fact **two different questions** a node can be asked, and both are needed.

The first is whether the archive is in its cache. The second is whether the closure is *materialised*, that is, whether its entry point exists on disk. A deployment with a shared store volume has the closure present without ever having cached an archive, so re-sending to it is wasted work; on a per-node store the entry point appears only after a real import. Either predicate alone is wrong for one of the two layouts.

Path-level difference. A closure is a set of store paths, and two environments that differ by one package share most of them. The client lists the closure's paths and asks the target which it lacks; if some but not all are missing, only those are sent, as a partial import that the receiver never caches as the whole closure. Adding one package to an environment a peer already held sent 7 of 43 paths, 206.0 MB reduced to 28.6 MB; adding `cloudpickle` to every environment cost a peer 3 of 39 paths, 0.43 MiB of a 444 MiB closure.

Signed transfer between daemons. After an upload, the worker pushes the closure to peers that cannot yet run it. This is not merely an optimisation: parallel work is only ever spilled to peers that can run the closure, so the push is what makes the cluster usable for the distribution described in Section 4.9. A peer that advertises the signed format receives one archive containing a signed binary cache — the signature records for every path and the compressed archives for the paths it lacks — produced by Nix's own copy mechanism and imported the same way, so the receiver's signature policy applies. Other peers receive the older unsigned export, which is also what a submitter uses for its own upload.

Bounded caches. Both closure caches are pruned least-recently-used to a cap of 8 GiB each by default, after every send and at daemon start. With the cap lowered to 50 MiB, the caches on one node went from 109 MiB to 28 MiB and from 768 MiB to 4 KiB, and the next job still ran on its peer, since what was evicted is derived data and was rebuilt. Workspace archives above 1 MiB and all results archives are compressed with Zstandard [34]; readers recognise Zstandard, gzip or plain archives by their leading bytes.

A transfer that can be seen. The client times packing the closure and sending it; on a terminal the send draws a progress bar, and otherwise it prints one line with the paths, bytes and time. A flag, `--cold`, sends the whole closure as if the node had none, skipping both questions and the path difference, so that a first run can be timed on a warm cluster. It cannot undo the submitter's own build or the node's earlier import, and the run summary marks such a send as cold; true first-run cost therefore comes from fresh machines. A signature refusal on import is no longer swallowed: the upload returns it and the client prints why the peer took no work.

4.8 Execution and Isolation

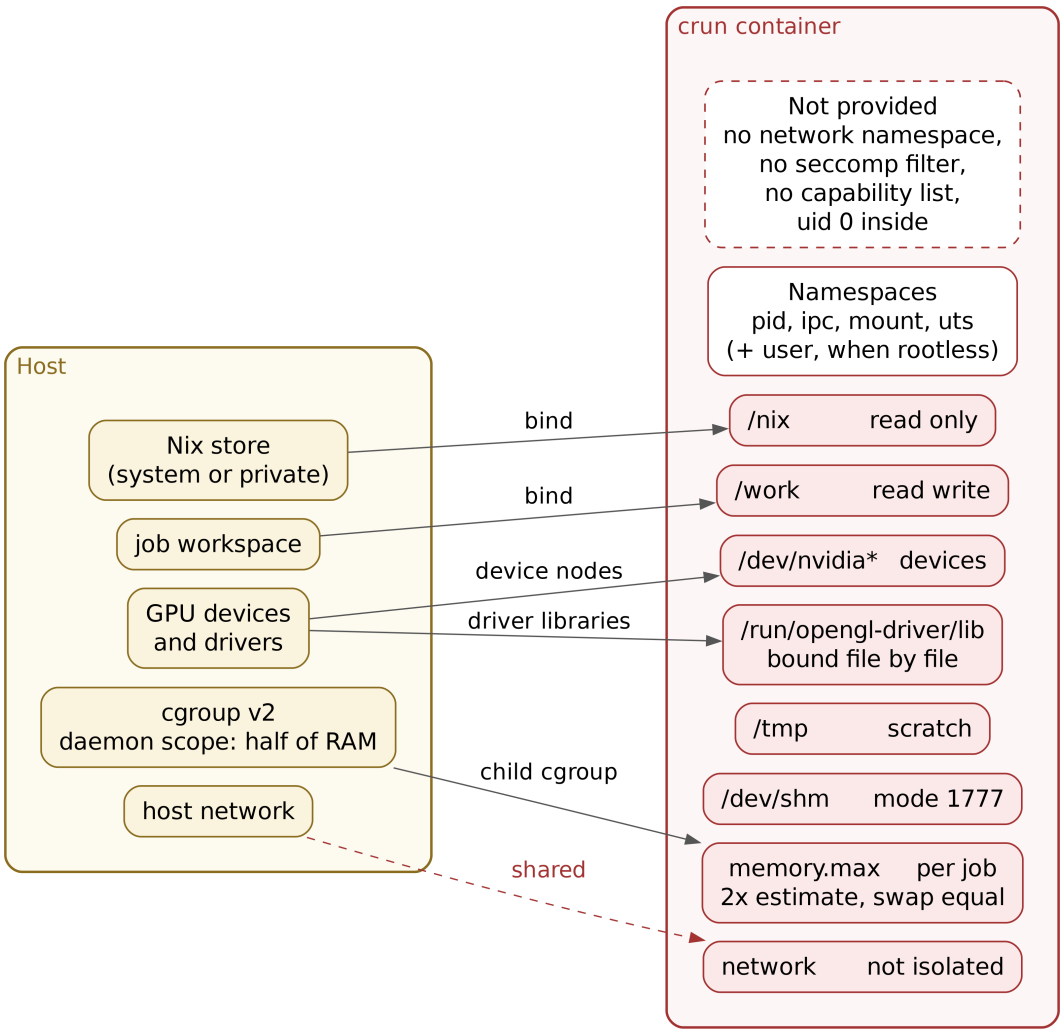


Figure 4.7: What the job can and cannot see. The absences are as deliberate as the mounts. Shared memory is mounted world-writable because Python's `multiprocessing` requires it, and the driver libraries are bound file by file so that the host C library cannot shadow the closure's.

Jobs run under `crun` in an OCI bundle [7] with a minimal read-only root: the store read-only, the workspace read-write, a private home, and temporary filesystems for scratch space. Shared memory is mounted world-writable specifically because Python's `multiprocessing` maps POSIX shared memory for its semaphores and fails without it.

GPU support mounts the device nodes and then binds the driver libraries *file by file* rather than binding the host library directory, because binding the directory would place the host C library ahead of the closure's on the search path. Without this, torch silently falls back to the CPU, which is indistinguishable from a working run except by its speed.

A memory ceiling for every job. Where the daemon has a usable cgroup v2 hierarchy (otherwise it logs once that jobs run without an enforced limit), each job runs in its own cgroup with a memory limit, and an equal swap limit, of twice its estimated requirement with a 512 MiB floor, clamped to what the node can grant but never below what admission approved. When the job is killed and the cgroup's out-of-memory counter has risen, the client is told the job ran out of memory and how to raise `--mem`, rather than seeing a bare exit code. A job with no estimate used to run with no limit at all; it is now capped at 4 GiB, with a note pointing to `--mem`, and the node still lowers that to what it can spare.

A memory ceiling for the daemon. Per-job limits covered only sandboxed jobs; parallel chunks run in warm worker processes in the daemon's own cgroup, and a pool benchmark on a development machine once drove it into a global out-of-memory condition in which the kernel killed an unrelated process. `pipedpeer start` therefore launches the daemon in a user `systemd` scope with a memory ceiling of half the machine and swap refused, where a user `systemd` is available. A daemon without one, such as one in a container, runs a watchdog instead: every second it sums the memory of the daemon's children and, past half the machine or the configured ceiling, kills the largest child and logs it, never the daemon. It stands down where a cgroup already sets a limit, and an allocation fast enough to outrun a one-second poll is not caught. A softer throttling limit was rejected on measurement: a 3 GiB allocation under a 1 GiB soft limit stalled at 998 MiB with 7,381 throttling events and no progress for minutes, which is worse than a failed job. Daemons on one machine, recognised by the kernel's boot identifier, subtract one another's reservations, so two of them cannot promise the same memory twice; on one machine with a host daemon and a container worker this changed the total promised from 7.24 GiB plus an unbounded daemon to 6.74 GiB and 1.97 GiB.

Without root. On a machine with no system Nix, set-up seeds a private store in the user's data directory and mounts it at `/nix` inside a user and mount namespace; `crun` is installed into the user's own binary directory when no package manager supplies it; and working files live under the user's state and cache directories rather than a RAM-backed temporary directory.

What the sandbox does not do. The job runs as user 0 inside the container and shares the host's network: there is no network namespace. The specification sets no seccomp filter and no capability list. The namespaces in use are process, IPC and mount, plus hostname where the kernel allows it; when `crun` runs without root it adds a user namespace itself. Isolation is the default but not mandatory: the `python` shorthand turns it off, and a node without `crun` or user namespaces warns and runs the job unisolated — unless it uses a private Nix store, in which case it refuses.

Results are diffed rather than copied wholesale. Only files created or modified during

the run are returned, with deletions carried in a manifest, so a job cannot overwrite the submitter's own sources. On extraction the submitter refuses any path that escapes its directory, and the worker skips such paths in an incoming workspace.

Output as it is written. The daemon used to split a job's output into lines, so a progress bar that redraws one line with a carriage return showed only its final state. It now forwards each read of up to 4 KB unchanged, carrying an incomplete trailing UTF-8 character into the next chunk, and the submitter prints the bytes as they arrive, so a progress bar on a remote node animates on the submitter's terminal.

Stopping a job. Until October nothing could stop a running job: the daemon never watched its client, and an interrupt left the script, its parallel chunks on peers and every training rank running. An interrupt, a termination signal or a closed terminal now makes the client send one cancel frame on the execution channel. The daemon signals the job's process group, or asks `crun` to, and kills it two seconds later if it is still alive; the final frame records the cancellation. The cancel reaches remote runs, `map` and every training rank. Chunks that a killed script had asked for stop as well: each carries its request's context, so when the request ends the local workers are killed by process group and forwards to peers are dropped, with nothing re-run. The coordinator records the lease as cancelled and does not retry. A connection that ends *without* a cancel frame — a crash, a lost network, a detach — never kills the job, which is what background mode builds on. In the rehearsal (containers on one laptop), peers stopped within **2** s of the interrupt: their pool counters showed the cancel at the next 1.3 s sample.

Background mode. The daemon keeps every frame of a job's output in a log in the job's directory, in the order a live client saw it. A detach flag, or the suspend key during a foreground run, closes the channel normally without a cancel frame; the client prints the job's short identifier and the command to reattach, and the prompt returns. `pipedpeer attach` replays the log, follows live without gaps or repeats, downloads the results at the end and prints the run summary; an interrupt during attach cancels the job. Two invariants make this safe. A lease follows its job: the execution request carries the lease identifier, and the sweeper keeps a lease bound to a running job past its expiry without renewals, because a detached or crashed submitter renews nothing, so a node's admission never thinks a busy slot free. And a detached job's directory is pruned 24 hours after it ends, while its output log is capped at 50 MB, keeping its newest half behind a note of what was dropped and never the final frame, so a job that prints without end cannot fill a disk. A job still marked running when its daemon starts is failed with a final frame saying the daemon restarted, so an attach ends rather than waits. A remote job survives its submitter vanishing and can be attached when the submitter returns; a parallel script cannot, because it runs on the submitter. Detach is offered for single runs, not for training rings or `map`.

4.9 Transparent Interception

This is the system's distinguishing feature.

Delivery. The shim is a Python file of 3,822 lines embedded in the binary. It is appended to the workspace archive under a hidden directory using an archive transform, so the user's project on disk is never modified, and the worker points the interpreter's module path at that directory. CPython imports it automatically before the user's first line. Nothing is installed on any node.

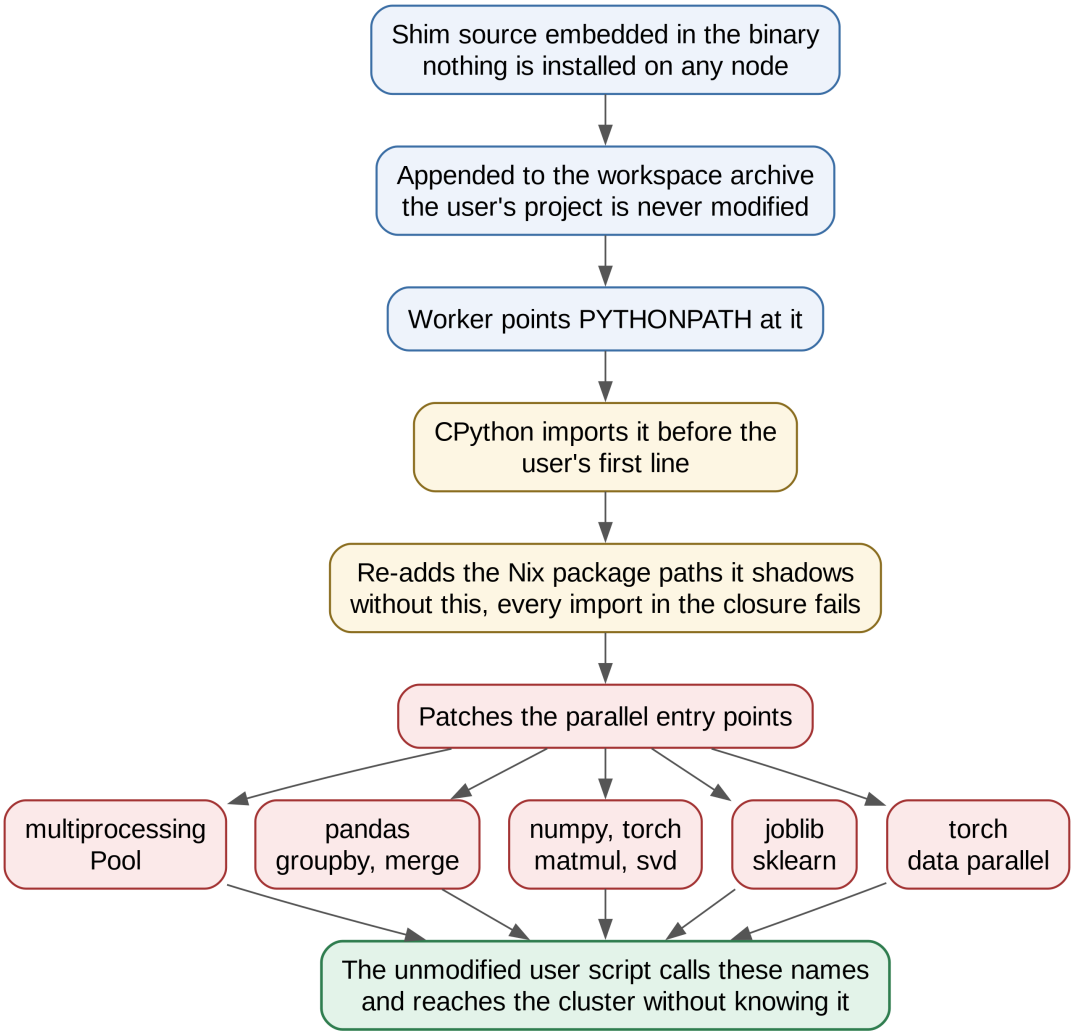


Figure 4.8: How the shim reaches the interpreter and what it replaces.

A hazard worth recording. Nix ships its own module of the same name, and only the first on the path is imported. The shim would therefore suppress the one that adds the environment's package directories, and every import in the closure would fail. The shim replays that work explicitly before doing anything else.

Patching is deferred. `multiprocessing.Pool` and `ProcessPoolExecutor` are patched

at start-up; NumPy, pandas, torch and joblib are patched on their first import, through an import hook, rather than when the shim installs itself. This matters: importing torch costs roughly 1.7 seconds, and paying it up front charged that cost to every job whether or not it used torch. Section 5.4 measures the difference.

Parallel maps in detail. `map`, `starmap`, `imap` and `imap_unordered` go to the cluster (`imap` lazily, in batches; `imap_unordered` currently returns results in order), while `apply` and `apply_async` stay local. The local pool's width is chosen from free memory and cgroup headroom, and the width the user asked for is ignored unless `PIPEDPEER_RESPECT_POOL_SIZE=`
A head of items runs locally first to measure the cost of one item, and a map whose remaining work is under half a second stays local. The kernel then has to reach the peer, and there are two ways for it to do so. A plain module-level function — including one defined in the script's `__main__`, which the earlier design could not ship at all — travels as its *source*, with the helper functions it calls and a pickle of the module data it reads. A lambda, closure, bound method, partial or decorated function, which has no source to send, travels *by value* through `cloudpickle`. Anything neither can carry, such as a function holding an open file, is counted as unshippable and runs locally. Every run can write a **receipt** counting the items that ran remotely and locally, the parts that failed, the calls that could not be shipped, and the node each part ran on, so that a claim about where work ran can be checked rather than inferred.

Sharing work by measured speed. On the daemon, a chunk is divided among peers by a makespan solver. Each device is modelled as a set-up time plus a throughput, fitted by least squares from the last eight parts it returned; an unmeasured peer is assumed to be as fast as this node, scaled by its core count, since assuming it slow would size its share below anything worth sending and it would never be measured. The solver finds the finishing time T at which the devices together complete the work, allocating each $(T - o_i)\, r_i$ items for set-up time o_i and rate r_i, and drops any device whose set-up time alone exceeds T: it "cannot start before the rest of the cluster finishes". A chunk posted by a shim on the same machine keeps no local share, because the local pool is already using those cores, and on a peer a chunk fans out across its cores. On a lab cluster limited to 8, 2 and 1 CPUs, dividing by headcount ran 1.6 times slower than not distributing at all; dividing by measured throughput declined the one-CPU worker and took a median of 2.87 s against 3.61 s by headcount.

Table 4.1: Intercepted primitives and their distribution strategies

Primitive	**Strategy**
`multiprocessing.Pool`, `ProcessPoolExecutor.map`	Measure a few items locally, then race local against remote chunks shared by measured speed
pandas `groupby().agg()`, `merge`, `join`	Hash shuffle on the key; bucket zero local
pandas `read_csv`, `read_parquet`	Out-of-core reads in 64 MB chunks cut at record boundaries
`numpy.matmul`, `dot`, `tensordot` (2-D)	Split into row blocks, concatenate at the origin
`numpy.linalg.svd`, `eig`	Whole matrix offloaded to one worker
`joblib.Parallel`, scikit-learn `n_jobs`	One batch per request; no cost model
torch `matmul`, `mm`	Row blocks, gated on size only
torch training (`--ddp`)	Gradients averaged by the lead daemon, weighted by samples

Interception replaces module functions, so the operator form `A @ B` is not intercepted for NumPy arrays. pandas operations are distributed only above 512 MB unless forced, because their arithmetic intensity is low.

Why the shuffle is exact. Hash partitioning on the key is the standard technique of MapReduce [6]; the choice here is to use only that path, never a combiner. Rows are bucketed by a hash of the grouping or join key, so equal keys always fall in the same bucket and therefore land on the same node. Every key is complete on exactly one node, so each node's aggregation is a *complete* aggregation and the origin combines the partials with a plain concatenation. There are no combiners, and consequently *any* aggregation specification works, including those that have no combiner form at all. Bucket zero is kept local.

What exactness means here. It means exactness of *values*, not of row order. A concatenation of shuffle buckets does not reproduce pandas' documented join ordering, which is why the join tests sort both sides before comparing; a script that depends on the row order of a `merge` result can therefore see a different order under interception. Grouped aggregation is unaffected, and its tests assert frame equality directly.

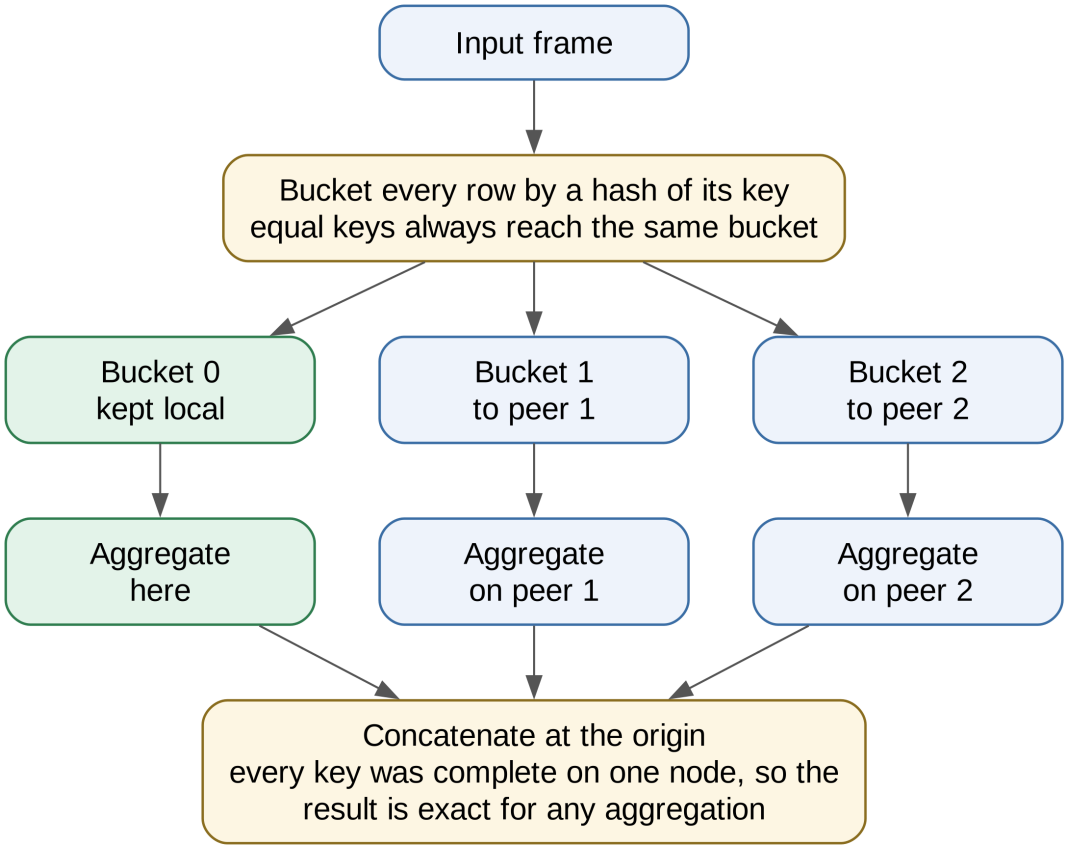


Figure 4.9: Hash shuffle. Correctness follows from key locality, not from combiner algebra.

Data-parallel training without a socket mesh. Rather than let torch build its rank-to-rank mesh, which requires direct connections on its own ports between every pair of ranks, each rank posts its gradient to the lead node's ordinary daemon port. That works on any network where the daemon port already works, including over the links of Section 4.2, which is the only assumption the rest of the system makes.

The daemon originally acted as a blackboard, returning every rank's gradient to every rank for the shim to average. Measured on two machines, that made training 21 times slower than one machine: 62.9 s against 2.9 s, with each step's reply carrying the whole ring. The daemon now *reduces*: gradients arrive as raw little-endian frames, are summed in double precision weighted by the number of samples behind each, $\sum_i n_i g_i / \sum_i n_i$, and one averaged gradient is returned. Gradients travel in half precision by default; an 8-bit encoding with one scale per tensor and the rounding error carried into the next step is available on request. On two machines over a home link, the same training run took 54.9 s on one machine, 69.4 s on two ranks in half precision with 36.2 s spent synchronising, and 56.1 s on two ranks in 8-bit, with final losses of 0.087495, 0.087497 and 0.087688. On that link two machines did not beat one: half precision was 26% slower, and 8-bit about level for a slightly worse loss. The run says so: each prints at exit whether the ring paid for

itself, counting distinct machines rather than ranks.

Choosing and refitting the ring. Ranks are chosen by measurement. Each candidate's throughput is probed, and the ring admitted is the prefix of the fastest that maximises $k \cdot r_k$ — the ring's worth if every rank ran at the slowest admitted rate — using one device kind only, GPUs when any are wanted and present. Each rank's shard and batch are then sized by its share. On a ring of 16, 8 and 2 cores, admission took the 16 and the 8, shared the work 66:34, and ran in 23.6 s against 25.4 s with equal batches. Every 20 synchronisations the daemon refits the shares from the step rates actually observed, applying them at the next epoch boundary if any share has moved by a tenth or more, and it can drop the slowest rank — never rank 0, never below two — when that rank adds more synchronisation time than computation. Throttling one rank from 8 CPUs to 1 and back moved the shares from 64:36 to 84:16 and then to 70:30. A round that a straggler has not joined completes without it after 45 seconds.

When a rank dies. Until October one failed rank failed the whole run, although the round already closed without it. The client now prints which rank died and on which host, tells the lead daemon, which stops waiting for that rank and finishes any round held on it, and the survivors take new shares at the next epoch boundary. The run fails only if no rank finishes or the user interrupts it, and the error names the real rank rather than a loop count. Samples are assigned to ranks by a weighted rendezvous hash [35] rather than the hash-modulo routing of the parallel map, which would move most samples when one rank left; the rendezvous hash keeps each survivor's share exact and over 95% of its samples in place. Losing rank 0's machine still ends the run, because its daemon holds the synchronisation, and a script that slices its own tensors instead of using a data loader is not re-spread.

When a machine comes back. A rank whose machine returns, or a new eligible node, joins the running ring at the next epoch boundary instead of sitting idle. The client watches the nodes and starts a rank there; the joiner skips ahead with empty passes over its data loader, loads the lead's model and optimiser snapshot through the lead daemon, and the shim's world size and rendezvous-hash shards grow to include it, so about $1/n$ of the data moves. The daemon's averaging group grows with it. Only data-loader scripts can grow; a script that slices its own tensors gets one line saying so. The refit now runs once per synchronisation and reaches every rank, where it used to hand new shares to one rank per cadence. Losing rank 0's machine still ends the run. In the rehearsal (two containers sharing one GPU) a returning rank joined the ring **4** s after its node's daemon started.

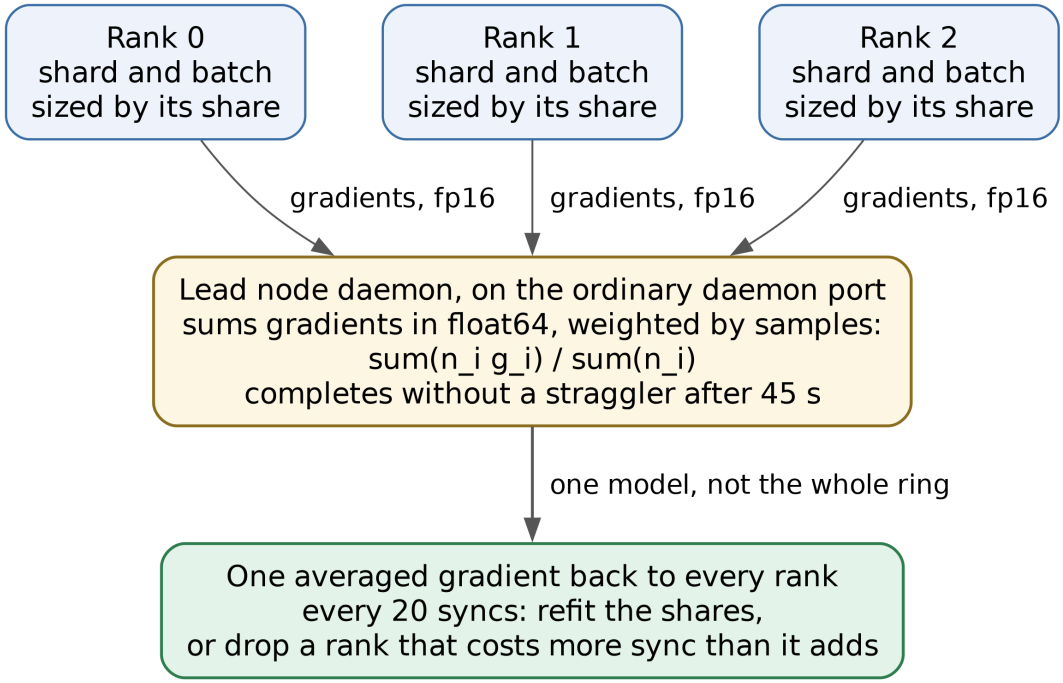


Figure 4.10: Gradient exchange over the daemon port. The lead daemon reduces the sample-weighted gradients and returns one average.

4.10 The Cost Model

Interception is only worthwhile when the cluster actually wins, so numerical work is gated. Writing b for the payload in bytes, ϕ for its arithmetic intensity in floating-point operations per byte, ρ for the number of times the data crosses the link, B for link bandwidth and K for the number of participating nodes, divisible work is distributed only when

$$\underbrace{\frac{b\phi}{R}}_{\text{compute locally}} > \underbrace{\frac{\rho\, b}{B}}_{\text{move it}} + \underbrace{\frac{b\phi}{R'K} \times 1.3}_{\text{compute remotely}} \tag{4.2}$$

where R is the local throughput, R' the remote throughput, and the factor 1.3 charges 30% overhead against the ideal parallel speedup. Matrix multiplication is priced at about 200 GFLOP/s with $\rho = 5$; pandas work at 1 GFLOP/s. Work that cannot be divided — an SVD or eigendecomposition, priced at about 1.5 GFLOP/s and sent whole to one worker — moves when the transfer costs less than half the local computation, since it frees the origin entirely.

The checks run in order. At least two nodes and 32 MB of payload are required. Then, **if the working set does not fit in local memory, the work goes to the cluster** whatever the transfer costs, and a matrix multiplication the cluster cannot take either

raises an error naming the size and the remedy rather than falling back to a local run that would exhaust the machine; this was added after an 8192-square product drove a 14 GB machine to 194 MB free and the kernel killed the desktop session. If any peer reports a GPU, R' is six times R, the figure measured on a small batch rather than the larger saturating one. The inequality is then evaluated at an optimistic 10 Gbit/s; if offloading loses even there, the work stays local without measuring anything. Otherwise bandwidth is *measured* by pushing 64 MB through the real dispatch path, cached for five minutes; if the probe fails, the answer is to stay local. Low-intensity pandas work ($\phi < 8$) is not distributed below 512 MB.

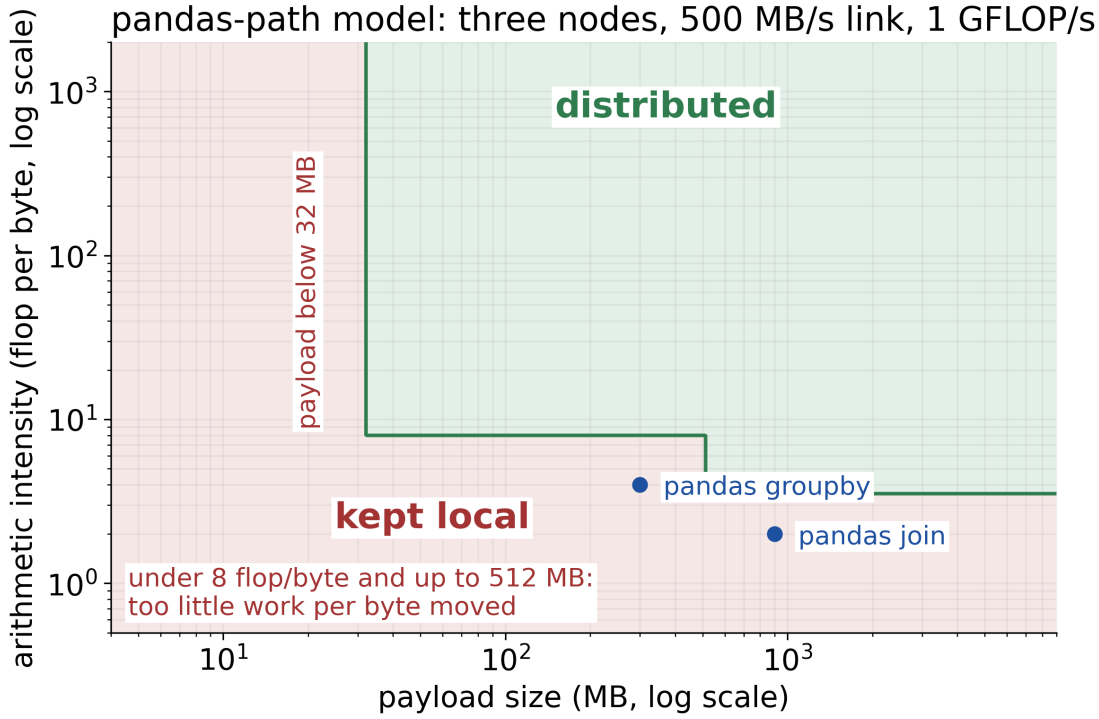


Figure 4.11: Decision regions of Equation 4.2 for the pandas path ($\rho = 1$, $R = R' = 1$ GFLOP/s) at three nodes on a 500 MB/s link. Grouped aggregation and joins have low arithmetic intensity, so they are distributed only above 512 MB. The NumPy path uses the same inequality with its own throughputs and is not drawn.

The consequence for dense matrix multiplication is counter-intuitive and worth stating. Local BLAS runs at roughly 200 GFLOP/s, which a CPU cluster's transport cannot beat at practical sizes, so the model keeps it local — measured against a four-core peer, local won by 25 to 100 times across four shapes and the model chose local every time. An earlier version of the project made this a law: the demonstration suite failed if a matrix multiplication was ever seen leaving the node. That was a fact about one rig written down as a rule, and it would have failed on the first cluster with a GPU. The rule is gone; matrix multiplication now ships when it does not fit locally, when a GPU peer makes the model favour it, or when forced, and the demonstration instead times the same product locally

and checks that the choice made was not the slower one. Singular value decomposition, two orders of magnitude slower locally, does ship.

4.11 Fault Tolerance

The invariant is that using the cluster is never worse than not using it. Five independent layers enforce it for parallel work; a training ring that loses a rank continues on the survivors (Section 4.9).

1. **The race.** Chunks are dealt alternately to the local side and the cluster, and results fill a slot table first-wins. Idle local cores re-run any chunk the cluster has not yet returned, and the call returns only once the local side has covered every item. A completely dead cluster therefore degrades to a plain local pool.
2. **Per-primitive fallback.** Every intercepted call is wrapped; any exception falls back to the original implementation. The one deliberate exception is a matrix multiplication that fits neither locally nor on the cluster, which raises (Section 4.10).
3. **Peer fallback.** A failed peer is skipped for the remainder of that chunk and the next candidate tried; if all fail, the work runs locally.
4. **Worker respawn.** A dead warm worker is restarted, falling back to a one-shot process if that fails.
5. **Rescheduling.** A failed job is placed again with backoff doubling from 2 seconds to a 2 minute ceiling. Tasks are never dropped; a cancelled one is not retried.

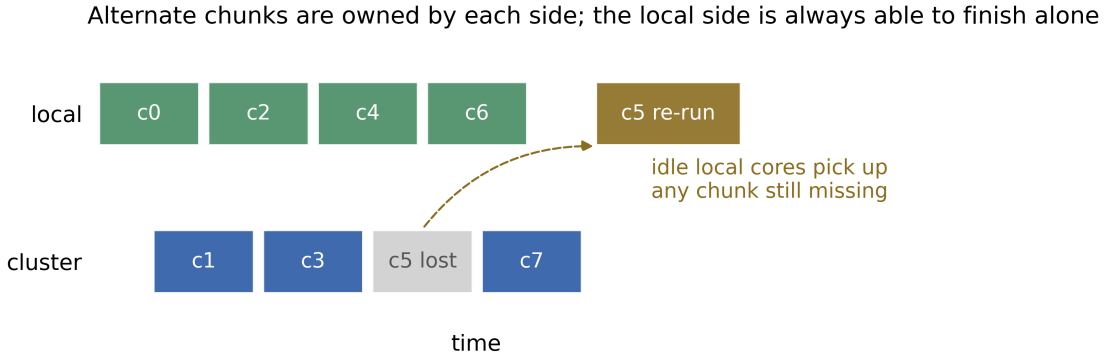


Figure 4.12: Racing local and remote work. The local side is always able to finish alone, which is what makes the guarantee unconditional.

Memory is bounded at the same time: a parallel-map request larger than a node can grant is forwarded to a healthier peer, one exceeding 40% of that memory is split into sequential chunks rather than refused, and a request body is capped at the smaller of 2 GiB and half the daemon's memory budget.

4.12 Resource Estimation

Tiers are consulted in order, the first that yields a value winning: an explicit override; the largest peak memory from the last five runs of the same script, plus 20%; or per-library baselines plus 100 MB for the interpreter. The estimator also has a tier based on input file sizes, but every caller currently passes no input files, so that tier is never reached. The default when nothing is known is 512 MB.

The historical tier closes a feedback loop. The worker samples the job's process tree while it runs and records the peak, and that measurement sizes the next run's admission request.

4.13 Seeing What the Cluster Does

Most of what a daemon did was visible only in its log. The demonstration work of October made it visible without adding a daemon endpoint beyond the two for background mode.

An events feed. `pipedpeer events` prints, in plain language, what this machine's daemon has done: peers joining and leaving, links made and the reason a peer has no path, leases, jobs started, finished and cancelled, environments sent and imported, keys trusted, and parallel work received and forwarded. Node identifiers are replaced by host names, repeated lines are collapsed, and a follow mode prints events as they happen. It reads the daemon's log, which mixes three formats from two logging libraries, with the project's own parser; the daemon gained one log line for each event that had none, and its log times now carry seconds. The feed is per machine: a sender sees what it sent and to whom, and a receiver who sent what.

Load on screen. Each node's heartbeat always carried its load; the node table and the terminal dashboard now show cores and CPU use, free memory, and the GPU's name, utilisation and memory, and the dashboard adds the last few events and a status line. A peer with no direct path is listed under the table with its reason, even when no peer is reachable. Joining redraws one line with the time left and the peers found, prints events above it, and on timeout lists the unreachable peers with three hints.

A real dry run. The check-only flag used to take a lease and cancel it. It now builds the environment where Nix is present, lists imports with no package, names the parallel calls in the source that can spread (a regular expression over the script, since a Python parser would need a Python on the submitter), and reports the memory estimate, the GPU and ring choice, where the job would go and how much of its closure that node holds, ending ready or not ready with the reasons. Without Nix the build is reported as skipped, not failed.

A run summary. Every run except a detached one ends with at most eight lines: where it ran, the environment build time, what happened to the closure (already present, a partial send, or a full or cold send with size and time), the script's time and peak memory, how its items spread across machines, the files that came back, and the total, or the moment it was cancelled. The spread comes from a receipt the shim prints on its error stream at exit, with items counted by where they ran; the client hides that line and parses it. The receipt now also counts the items the executing machine ran itself, including a chunk it re-ran after a peer failed.

4.14 Test Workloads

Ten dispatch scripts exercise placement and concurrency. Seven demonstration workloads cover a standard-library parallel map, a map over a lambda, scikit-learn with `joblib`, NumPy linear algebra, out-of-core pandas, torch data-parallel training and file synchronisation, and a walkthrough runs them across three machines on three unrelated networks. A three-node container lab and a lab with unequal CPU quotas provide the single-host environments used in Chapter 5. Benchmark harnesses measure parallel maps against hand-tuned pools, the work-sharing policy, distributed training, matrix multiplication decisions, closure transfer and interception start-up; a cross-network acceptance script checks that peers on different networks link directly; and two scripts audit the tests themselves (Section 5.2).

Three rigs were added in October. A NAT lab places each node on its own private network behind a router container — two cone and two symmetric translators — with the introducer on a public network, and asserts that cone pairs punch and take work, that a cone–symmetric pair links by the birthday search, that the symmetric pair is reported unreachable with its reason, and that a multi-user Nix node set up once takes work from a node that joins later without a prompt. A venue-shaped lab puts containers on one bridge network as a college LAN, with the host's own daemon as the GPU machine and introducer and one container on multi-user Nix standing in for the lab's Ubuntu machines; a two-container lab shares the host's one GPU between two nodes for the training ring. One script combines every kind of intercepted work, and five scripts written the way a student would write them stand in for code the system has never seen.

Chapter 5

Results and Discussion

5.1 Test Bed and Sources of Evidence

Three sources of evidence appear in this chapter, and every figure states which it comes from. Raw output for each is kept in the project's `docs/benchmarks/raw` directory.

- **October 2026 runs.** Run for this report on 2 October 2026 at commit `4c91ef3`, on one host: Fedora 43, Linux 6.19.13, an Intel i5-13420H with 12 hardware threads and 15 GiB of memory, Go 1.25.9 and Python 3.14.4, with three worker daemons in containers on ports 38081–38083 and the submitting daemon on 38080. On this host the user's home directory is itself a version-controlled repository, which triggers a build-directory defect listed in Chapter 6; the cache directory was therefore pointed outside it for every run.
- **August 2026 runs.** The previous measurement pass, on 25 August 2026, on a different host with 20 logical cores. Quoted only where the code measured has not changed or where the October run did not repeat it, and labelled as such.
- **Recorded measurements.** Measurements between real machines, taken by hand between 26 and 29 August 2026 and recorded in the commit message of the change each one justified. They are quoted verbatim, with the commit, in the raw directory. They were not repeated for this report.
- **12 October 2026 runs.** On a second host (Fedora 44, 16 hardware threads, an RTX 3060 laptop GPU): the re-run of the idle interception measurement at commit `08ce1d4`, the test suite after the demonstration round, and the rehearsal of the demonstration in the venue-shaped container lab at commit `31b5364` (first-pass figures at `8e14b9b`) and a two-container GPU lab at `1519952`. Labelled as such.

One caveat governs the laboratory results. All three workers are containers on one machine and share one processor, one memory system and the loopback interface. The laboratory measures correctness, overhead, cache behaviour and fault handling. It *cannot* measure multi-machine speedup, because moving processor-bound work between containers on one host cannot make that host faster. The only speedup figures in this chapter are recorded measurements between real machines (Section 5.8), and they carry their provenance.

5.2 Functional Verification

The full suite was run with every optional Python library present, using the CPU build of torch that the continuous integration job installs.

Table 5.1: Test suite results (October 2026)

Outcome	**Count**	**Notes**
Passed	615	every one of the 36 packages that has tests
Failed	0	
Skipped	2	integration tests that need an explicit opt-in flag

The same suite on the same host *without* torch installed gave 607 passed, 3 failed and 7 skipped. The three failures are tests of the weighted training sampler that raise when torch is missing rather than skipping; this is recorded as a defect of the tests rather than of the code they test; they now skip.

After the October rounds (the leftover fixes and the demonstration work), the same suite gave **822** passed, **0** failed and **7** skipped across **38** packages: two opt-in integration tests and five that depend on how Nix is installed on the host. New tests cover, among others, the hello's binding to its session, the secret going only to hosts that prove it, a cancel frame ending a running job within three seconds, a detached job running to completion with every output frame logged, attach replaying a running job with no duplicates, a job-bound lease outliving its expiry while the job runs, a GPU-less submitter left out of a GPU ring, and an import followed through a helper module.

Two further checks were run, both of which exist because a green test suite had previously been misleading in this project. The first, **the skip check**, fails if any of fifteen named tests that exercise distribution across nodes was skipped rather than run; all fifteen ran. The second, **the mutation audit**, breaks one feature at a time in the source — 95 of them, from the sample-weighted gradient fold to the router-mapping checks — runs the tests named for that feature, and requires them to fail. All 95 mutations were caught, and the audit restores the source exactly afterwards.

Table 5.2: Examples of what the tests establish

Test	**Property established**
Hash shuffle matches local	grouped aggregation is frame-equal to local pandas; the four join types are equal up to row order and index
Pool distributes `__main__` functions	a kernel defined in the script itself executes on another node
Race with dead remote	every remote chunk failing still yields correct results
Gradient synchronisation	two live ranks; each rank's gradient is exactly the mean
Weighted gradient fold	the daemon's average is $\sum n_i g_i / \sum n_i$
Routing around an unreachable peer	a peer with no direct path is never offered to the scheduler
Double-NAT mapping not published	a router mapping whose address differs from the reflexive one is withheld
No signature switch-off	set-up never writes `require-sigs = false`

5.3 Environment Cache

The same trivial job was submitted five times to the laboratory (October 2026). Each run's own log records which worker it was placed on and what was sent.

Table 5.3: Five submissions of one environment (October 2026)

Run	**Seconds**	**What the log records**
1	23.41	first build; whole closure sent to worker A
2	8.08	placed on worker B, which did not yet hold it; whole closure sent
3	0.86	worker A; closure already present, nothing sent
4	0.83	worker A; nothing sent
5	0.82	worker A; nothing sent

A submission whose closure is already present costs a median of 0.83 s against 23.41 s for the first, which also built the environment. The second run is reported as it happened: it landed on a worker that had not yet received the closure, so the whole closure crossed again. The August measurement of the same harness on the other host gave 12.00 s cold and 0.65 s warm.

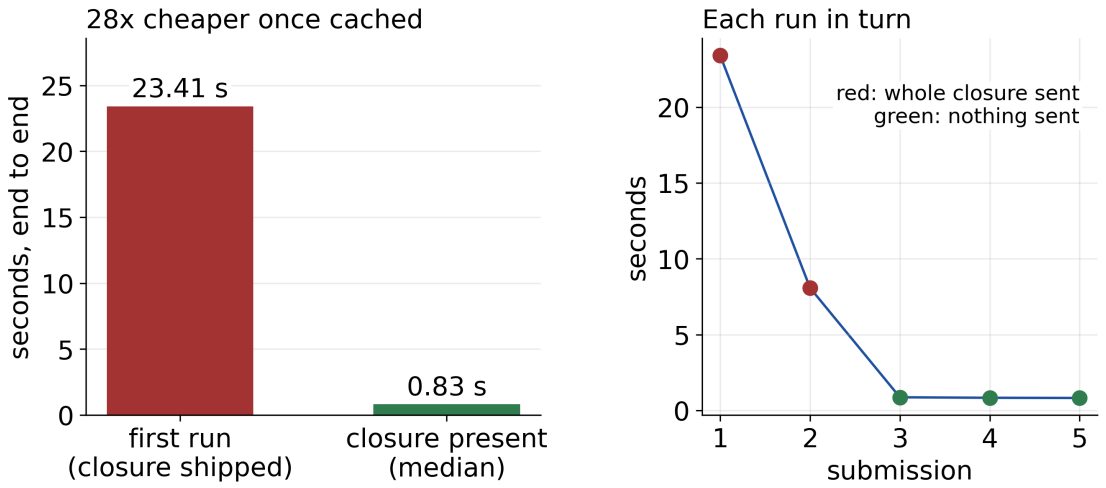


Figure 5.1: Left: first submission against the median of the submissions that sent nothing. Right: every run, coloured by what its log says was sent.

Two recorded measurements between real machines show the path-level difference that the laboratory does not exercise: adding one package to an environment a peer already held sent 7 of its 43 store paths, 28.6 MB instead of 206.0 MB; and adding `cloudpickle` to every environment cost a peer that held the rest 3 of 39 paths, 0.43 MiB of a 444 MiB closure.

5.4 The Cost of Leaving Interception On

Table 5.4: Whole-job time of a 48-item parallel map with interception enabled and disabled (October 2026, five runs each)

Configuration	Seconds	Notes
Interception disabled	0.94, 0.66, 0.65	three clean runs; two others failed once and were retried (9.66, 3.25)
Interception enabled	0.97 – 1.10, median 0.98	every run sent 18 of 48 items to two other daemons

This harness was meant to measure interception on a workload the cost model declines. At this commit it does not: in every enabled run the shim judged the map worth distributing and sent 18 of its 48 items to two other daemons. On one host that can only add cost, and it did — about 0.3 s over the two fastest disabled runs, which ran on a different worker. Two of the five disabled runs hit an execution failure (the worker's output stream closed early) and were rescheduled by the coordinator, which is why their times are an order of magnitude larger; they are recorded as a defect, not as variation. The idle cost of

interception is therefore not measured by this table; the start-up measurement below is the direct evidence for it. (August's version of this harness on the other host gave 0.85 s disabled against 0.93 s enabled.)

The measurement was repeated on 12 October with a workload small enough that the cost model declines it: a 16-item parallel map whose tail is 0.01 s of work against a 0.5 s dispatch floor. The harness now fails any run whose receipt shows a dispatched item, and a forced run of the same workload does dispatch, so the decline is the cost model's choice and not a dead cluster.

Table 5.5: Whole-job time of a declined 16-item parallel map (12 October 2026, second host, five runs each)

Configuration	**Median**	**Range**	**Notes**
Interception disabled	0.678 s	0.671 – 0.852 s	
Interception enabled	0.796 s	0.747 – 0.797 s	plus one cold first run of 7.03 s; every receipt shows nothing dispatched

Interception that declines therefore costs about 0.12 s per job: the measurement of the head of the map plus the decision.

Interpreter start-up for a job that imports nothing heavy is measured directly.

Table 5.6: Interpreter start-up overhead for a job that imports nothing heavy

Configuration	**Slowdown**	**Within the 3× budget?**
Eager patching (August, before the fix)	20.0×	no
Deferred patching (October)	1.24×	yes

The check behind these figures now runs in continuous integration on pushes to the main branch and on pull requests into the main and development branches, alongside the skip check and a never-slower pool benchmark.

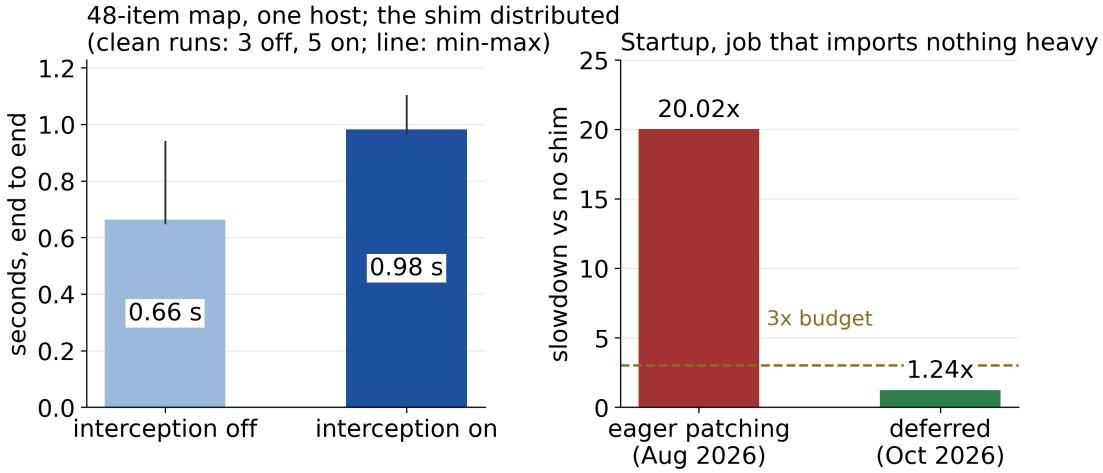


Figure 5.2: Left: the parallel-map job of Table 5.4, clean runs only (failed-and-retried runs excluded). Right: interpreter start-up, with eager patching (August) and deferred patching (October).

5.5 Work Crosses Node Boundaries

With distribution forced, every one of five runs of a 48-item parallel map reported, in its own receipt, 18 items completed on three other daemons and the rest locally, with an identical checksum each time. The NumPy workload moved its 34 MB singular value decomposition to one worker, as the unsplit cost model allows. Its matrix product is written with the `@` operator, which is not intercepted (Section 4.9), so it ran locally without consulting the model at all.

The project's own pool benchmark, run against the laboratory, is the honest counterpart: plain Python took 0.17 s, a hand-tuned pool 0.17 s, and the untouched script under PipedPeer 0.24 s with 12 of 48 items run on the cluster. On one host, distribution can only add overhead, and it did — within the project's 3× never-slower budget. Section 5.8 reports the same benchmark between real machines.

5.6 Fault Tolerance

The fault-injection harness fans an 80-item parallel map across the laboratory, waits until a worker is holding chunks, kills it, and requires the run to finish correctly with work recorded as having left the submitting process. It was strengthened after August, when it was found to prove nothing: its kernel could not then be shipped, so the local pool did all the work and a kill changed nothing.

In the October run it failed. The job was placed on the submitting host, which already held the closure, so nothing was sent; the freshly created workers never received

the closure; and since parallel work is offered only to peers that can run it, the shim ran all 80 items locally. No worker ever held a chunk, and the harness reported failure by its own rule. The likely cause is that the closure is pushed to peers only after an upload, so a job placed on a node that already holds its closure never seeds the others; this reading of the log has not been verified further.

The guarantee that remains evidenced is the one the tests establish: every remote chunk failing still yields correct results, and a run with a dead remote completes (Table 5.2). The August run of the earlier harness completed correctly after a worker was killed, but that harness was later shown not to have distributed any work, so it is not cited as evidence here.

5.7 Sandbox Cost

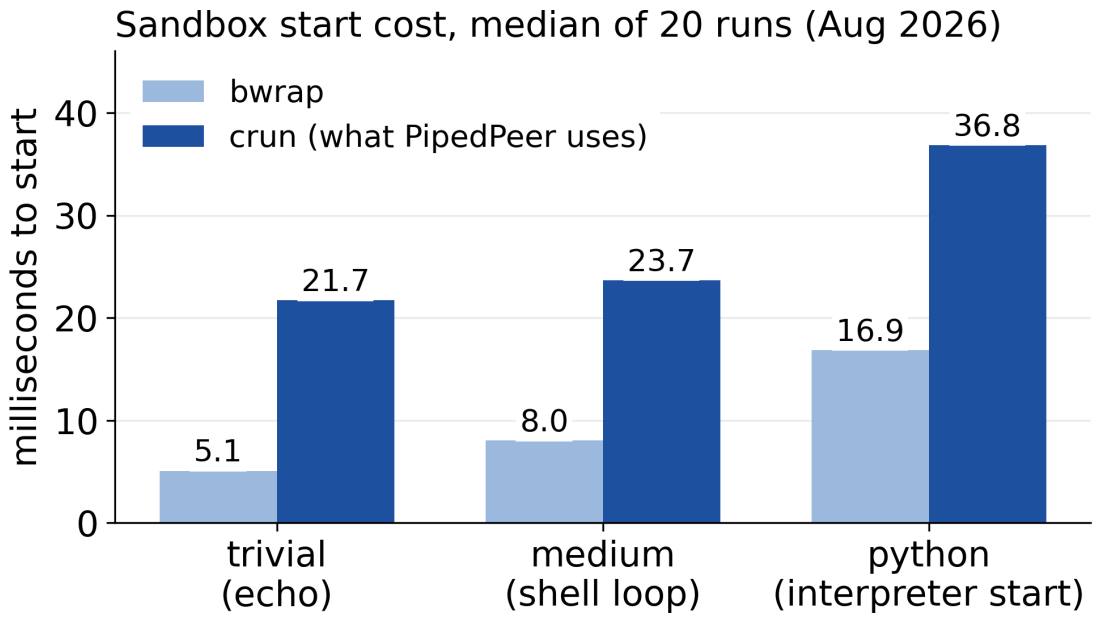


Figure 5.3: Sandbox start cost, median of 20 iterations (August 2026).

Measured in August and not repeated: the OCI runtime costs 15 to 20 ms more per job than the lighter alternative, small beside a warm job of under a second, and its device model is what makes GPU passthrough possible. The per-job memory cgroup added since was not timed separately.

5.8 Measurements Between Real Machines

Table 5.7 collects the multi-machine measurements recorded during development. Each was taken by hand, a small number of times, on the machines and links available —

the development machine, a second machine, a public cloud host as introducer, and the laboratory — and recorded in the commit it justified. They are evidence of behaviour, not a controlled experiment. One earlier single-sample figure in the project's history, an 8-bit encoding said to cost "about 5%" of the final loss, was found on repetition to cost 45%, and its replacement (with one scale per tensor) is what is quoted here.

Table 5.7: Recorded measurements between real machines (August 2026)

Measurement	**Result**	**Commit**
Parallel map, 20-core machine and 16-core peer over Tailscale, plain Python vs. PipedPeer	5.8–6.4 s vs. 4.9–5.4 s; 116 of 256 items on the peer	`fca0107`
Same, before chunks fanned out across the peer's cores	6.4 s vs. 8.5 s (slower)	`fca0107`
Work split on an 8:2:1-CPU laboratory, headcount vs. measured	median 3.61 s vs. 2.87 s	`8c9284c`
Training, two machines, daemon as blackboard	62.9 s vs. 2.9 s on one machine	`145db30`
Training, two machines on a home link: one machine / two ranks fp16 / two ranks int8	54.9 / 69.4 / 56.1 s; loss .087495 / .087497 / .087688	`364b3d9`
Ring of 16, 8 and 2 cores: equal vs. measured batches	25.4 s vs. 23.6 s; 2-core rank not admitted	`2fc3914`
Three-rank ring vs. the one machine admission picks	36.4 s vs. 6.9 s	`2fc3914`
Same model and batch, GPU vs. CPU	1.5 s vs. 55.0 s	`2da0857`
Shares after throttling one rank 8 CPUs to 1 and back	64:36, then 84:16, then 70:30	`ac19310`
Closure difference, one package added	7 of 43 paths; 206.0 MB to 28.6 MB	`193e1bf`
Peer reached with the introducer suspended	0.03 s	`42c2426`
Introducer traffic during a verification run	0.2 MB, against about 30 MB between peers	`42c2426`
Phone-tether NAT: ports seen by two reflectors; punch to a home router	38471 and 27486; 400 packets each way, none received	`fe43933`
Signed closure on a host requiring signatures	imported when trusted; refused when the key was removed	`cf2e3ae`
Daemon memory ceiling, 3 GiB allocation under 1 GiB	killed in seconds, OOM recorded against the scope	`96c7f35`

Read together, they support modest claims. A parallel map ran faster across two machines than on one, though well short of what the core counts allow, and only after chunks were made to fan out across the peer's cores. Dividing work by measured speed beat dividing it by headcount on an uneven cluster. Two-machine training on a home link of about 54 Mbit/s did not beat one machine — half precision was 26% slower and 8-bit about

level — while the same job on one machine's GPU took 1.5 s against 55.0 s on its CPUs; the system reports at the end of each run whether the ring paid for itself rather than leaving the user to discover it. Peers on different networks linked directly and stayed linked with the introducer suspended, and the introducer carried a negligible share of the traffic; the network types involved in that run were not recorded. Not measured: whether two GPUs in one machine train faster than one, which the placement supports but no available machine could test.

5.9 Scale of the Implementation

Table 5.8: Implementation size at commit `4c91ef3`

Measure	**Value**
Go source, excluding tests	32,049 lines across 83 files
of which the embedded Python shim	3,822 lines
Go tests	23,100 lines across 99 files
Packages	38 (37 internal, plus `main`)
HTTP routes on the daemon	22
Tests run	617 (615 passed, 2 skipped)
Mutations in the test audit	95, all caught

The August figure of 24,679 lines counted test files together with source; on that basis the October total is 55,149 lines. After the 12 October rounds the same counts are **42,424** lines of source in 114 files, **30,767** lines of tests in 154 files (at `cc56753`) and **829** tests run.

5.10 Rehearsing the Demonstration

Before the demonstration in a college laboratory, the whole of it was rehearsed on one laptop: the venue-shaped container lab for the processor steps, and two containers sharing the laptop's one GPU for the GPU step and the training ring. The containers share one processor, so these figures show what the steps cost and that each works, not speedup. A cold run is the first ever on fresh containers; a cached run follows it.

Table 5.9: Demonstration steps, first run and cached (12 October 2026, container labs on one laptop)

Step	**Cold**	**Cached**
Live join, until connected	**1.8** s	—
Headline parallel map	**43.5** s	**17.0** s
Map over a lambda, `--cold` then cached	**20.5** s	**4.8** s
GPU job submitted from a GPU-less node	**264.8** s	**7.6** s
Two-rank GPU ring on one shared GPU	**630.3** s	**9.5** s
Combined script	**664.6** s	**26.7** s
Five student-style scripts	**35–126** s each	**19.6–69.5** s each
Unseen script's environment, without and with common packages	**54** s	**4** s
Every script at once	**412.0** s, all 8 jobs ok	

The first runs of the headline map and the GPU job are from the first rehearsal pass (`8e14b9b`), since the second pass found those steps warm; every other figure is from the second pass (`31b5364`), the cached student runs and the ring's cached run from a re-check at `cc56753`, and the ring's first run, the combined script and the finale from a last pass at `9949ad0` (the ring's first run took 635 s at `cc56753`). A join has no cached form. The finale ran its eight jobs with the demo's `PIPEDPEER_REMOTE_WAIT=10m` and with the lab's containers resized to 6.0 GB of memory in total; at 1.2 GiB each, three of its jobs fitted no node.

Table 5.10: Failure drills in the rehearsal

Drill	**Result**
Interrupt during a parallel map: time until peers stop	under **2** s
Terminal closed during a run	stopped everywhere, as an interrupt
Detach, then attach while running and after it finished	followed live; replayed in **1.0** s once finished
Submitter's network removed during a remote job, then attach	**200** s outage: ran once, results back **15** s after reconnect
A node stopped during a run	same checksum, **16.5** s against **17.0** s undisturbed
A training rank killed during a run	survivor finished all 80 epochs, **90.5** s
A training rank's machine back during a run	rejoined at the next epoch, **4** s after its daemon started

5.11 Comparative Analysis

Table 5.11: PipedPeer against existing systems. The lower block is where PipedPeer is partial or the weaker system.

Property	**Piped-Peer**	**Spark**	**Ray**	**Dask**	**BOINC**	**ssh**
What it does differently						
Runs unmodified Python	yes	no	partial	partial	no	yes
No cluster to configure	yes	no	no	no	no	partial
No head or master node	yes	no	no	no	no	yes
Reproduces the environment	yes	partial	partial	partial	partial	no
Joins NATed machines, no relay	yes	no	no	no	no	no
Survives losing a node	yes	yes	yes	yes	yes	no
Where it is partial or behind						
Authenticated and encrypted	partial	yes	partial	partial	yes	yes
Sandboxes each job	partial	no	no	no	partial	no
Runtimes other than Python	**no**	yes	partial	no	yes	yes
Proven in production use	**no**	yes	yes	yes	yes	yes

	PipedPeer	Spark	Ray	Dask	BOINC	plain ssh	
Runs unmodified Python	yes	no	partial	partial	no	yes	what it does differently
No cluster to configure	yes	no	no	no	no	partial	
No head or master node	yes	no	no	no	no	yes	
Reproduces the environment	yes	partial	partial	partial	partial	no	
Joins machines behind NAT, no relay	yes	no	no	no	no	no	
Survives losing a node	yes	yes	yes	yes	yes	no	
Authenticated and encrypted	partial	yes	partial	partial	yes	yes	partial, or behind
Sandboxes each job	partial	no	no	no	partial	no	
Runtimes other than Python	no	yes	partial	no	yes	yes	
Proven in production use	no	yes	yes	yes	yes	yes	

Figure 5.4: The same comparison in matrix form. The criteria in the upper block were chosen because they are what PipedPeer set out to change; the lower block is included so the table is not simply a list of its own features.

The upper block should be read for what it is: the criteria on which PipedPeer was designed to differ, so it wins them by construction. The lower block is the more informative half. Authentication and encryption are now present but partial — optional, trust-on-first-use on a local network, and without per-user authority (Section 6.3) — where Spark and BOINC offer established mechanisms. Isolation is partial in a different sense: memory is limited per job, but the sandbox shares the host network and can be skipped on a node without `crun`. PipedPeer is restricted to one language and has no production history: the others have years of deployment, and this system has a laboratory and a handful of machines.

Spark, Ray and Dask are also considerably more capable at scale, with mature schedulers, richer fault tolerance and far broader operational tooling. The honest summary is that PipedPeer occupies a different point on the curve rather than a better one: it trades maturity and scale for the elimination of set-up cost, code change and network configuration. BOINC [3] is closest in spirit, having pioneered volunteer computing, but it is a platform for specific projects each with its own server infrastructure rather than a general executor.

5.12 What Was Not Measured

Recorded explicitly rather than left as gaps.

Table 5.12: Measurements not made for this report, and why

Measurement	**Why it is absent**
A controlled, repeated multi-machine benchmark	only one machine was available for this report; the recorded figures of Section 5.8 are the substitute
Fault injection that actually distributes	the October run did not distribute (Section 5.6)
Hole-punch success rate across many networks	each link was observed individually; no survey of NAT types was made
Two GPUs in one machine	no available machine has a second usable GPU
Peer discovery latency	containers share the host's loopback, so the figure would not be meaningful
The seven demonstration workloads end to end	not run for the October pass; rehearsed in containers on 12 October (Section 5.10)
Speedup, wireless networks, two physical GPUs and a college network's rules	the rehearsal ran in containers on one laptop; these come only from the venue

Chapter 6

Conclusion and Future Work

6.1 Conclusion

PipedPeer runs an unmodified Python script on other machines with no cluster to configure, reproduces its environment exactly, distributes the parallel work inside it without the user touching the source, and joins machines on different networks with one command and no relay. The contributions are:

1. **The store path as a cluster-wide cache key, and the store path as the unit of transfer.** Pinning an exact package set makes an environment's content-addressed name identical on every machine and on any day, so a worker that holds an environment pays nothing for it (23.41 seconds cold against 0.83 seconds warm in the container lab), and a worker that holds part of it receives only the paths it lacks (7 of 43 paths, 86% fewer bytes, between two real machines).

2. **Transparent interception with work shared by measurement.** The parallel primitives a script already uses are substituted at import time. Kernels travel as source or, for lambdas and closures, by value. Parallel maps are divided by a makespan solver over measured peer rates, and numerical work moves only when the cost model says moving it is cheaper than computing it.

3. **Joining across networks without a relay.** An introducer that never carries data, router port mapping, a candidate race over one shared socket, a birthday-collision fallback for the hardest pair of NATs, and mutually authenticated QUIC links that the rest of the system sees as ordinary nodes.

4. **Placement without a scheduler.** Hard filters followed by a bounded score, executed in the submitting client, with a three-phase lease providing atomic reservation and crash recovery through expiry, against memory budgets that every daemon on a machine respects together.

5. **A degradation guarantee that is tested, not asserted.** Five independent layers ensure that a remote path has a working local path beneath it; the tests show a run completing correctly when every remote chunk fails; and a mutation audit that disables 95 features one at a time is caught by the test suite in every case. The end-to-end fault-injection run did not distribute work on this report's laboratory and remains to

be repeated.

The part most worth defending is still the refusal to distribute. Interception that shipped everything it could intercept would be slower than plain Python on most workloads. The project repeatedly measured its own features and kept them off, or changed them, when the measurement disagreed: gradient averaging moved into the daemon when the blackboard design made training 21 times slower; a coarser synchronisation interval that ran ten times faster stays opt-in because it trained a worse model; work is divided by measured speed because dividing by headcount was slower than not distributing; and a rule that dense matrix multiplication must never leave the node was replaced by a check that the choice made was not the slower one, because the rule would have been wrong on the first cluster with a GPU.

6.2 Limitations of the Current System

- **No relay.** Two machines between which no direct path exists — for example two carrier networks that both allocate a new port per destination — cannot work together. Such a peer is reported and routed around.
- **Python only.** Other runtimes would need their own environment packaging and their own interception layer.
- **Linux workers.** The sandbox, cgroups and the store are Linux-specific. The private-store seed is built for x86_64 only.
- **A parallel script needs its submitter.** A remote job survives its submitter vanishing and can be attached later (Section 4.8), but a script whose parallel work is spread runs on the submitter, so that machine must survive the job.
- **A training ring needs rank 0's machine.** A ring continues without any other rank, but rank 0's daemon holds the synchronisation, and re-spreading a dead rank's data needs a data loader.
- **Dependencies outside the package set.** An import that neither the table nor the package set knows is reported before the run rather than dropped silently (Section 4.6), but the job still needs the user to name a package for it.
- **Interception gaps.** Only the `map` of a process-pool executor is intercepted, not `submit`; grouped aggregation on categorical keys is never split; and numerical work under 32 MB never leaves the node, by design.
- **A per-machine view.** The events feed shows one machine's daemon; there is no cluster-wide view.

- **A partial sandbox.** Memory is limited per job, but the job runs as user 0 inside the container with the host's network, without a seccomp filter, and isolation is skipped with a warning on a node lacking `crun` or user namespaces (Section 4.8).
- **Measurement.** The single-host lab cannot measure speedup, and the multi-machine figures in this report were recorded by hand during development rather than by a repeated, controlled experiment (Section 5.12).

Defects found while preparing this report

Writing the measurements down found defects twice. The August pass found three, all since fixed: the shim imported torch at start-up, taxing every job about 1.7 s; the sandbox benchmark gave neither sandbox a root filesystem, so every one of its runs had failed; and the laboratory scripts could not be run twice. The shim start-up check now runs in continuous integration.

The October pass found the following, which are recorded here rather than corrected, since this report documents the system as it stands:

1. When the user's home directory is itself a version-controlled repository, the build directory — moved from the system temporary directory into the user's cache directory — falls inside it, Nix refuses the generated flake as untracked, and the coordinator retries the job indefinitely. The measurements in Chapter 5 were taken with the cache directory pointed outside the repository.
2. Three tests fail, rather than skip, on a machine without torch.
3. Two of five runs of one benchmark failed with the worker's output stream closed early (`stdout scanner: read |0: file already closed`) and were rescheduled; the cause was not investigated.
4. On a freshly created laboratory, a job placed on the submitting node, which already held its closure, never pushed the closure to the workers, so its parallel map stayed local and the fault-injection harness could not run (Section 5.6). The cause is inferred from the log and not verified.
5. Reading the code for this report raised further suspected defects that were not exercised: compressed parallel-map items whose flag the daemon does not carry forward, a memory-constrained reply path that answers in a different framing from the one the shim reads, receipts that count forwarded work as local, removal of one internet peer deleting every loopback forwarder, the shared secret being attached to subnet-wide health probes, and the firewall-opening commands running without elevated privileges. They are listed for the developers in the repository's documentation notes.

Most of these have since been fixed. The build now runs outside the user's repository; the three tests skip without torch; the fresh-laboratory failure was traced to every worker refusing the broadcast closure for want of a trusted key and fixed as Section 4.3 describes; and the compressed-item flag, the reply framing, the receipt's count of locally run items, the forwarder removal, the secret sent to subnet probes and the unprivileged firewall commands were each fixed in a commit of their own. The early-closed output stream was the daemon reaping a job before its output readers had drained the pipes; it now waits for them.

6.3 Security

The previous version of this report stated that PipedPeer had no authentication, no authorisation and no transport encryption. That is no longer true, and Section 4.3 describes what replaced it: a shared secret on every route, sent only to hosts that prove they hold it; pinned TLS on the daemon port; QUIC links across networks authenticated by node keys, with each hello bound to its session and the secret; and closures signed by the node that built them, whose keys reach system Nix through a root-owned helper that can do nothing else.

What remains is still significant and is stated plainly. The secret is optional, and a daemon without one accepts any request that reaches it, which means arbitrary code execution for anyone who can reach the port. With a secret, every holder of it can run code on every node; there is no per-user authority, and jobs themselves can read the secret. TLS on a local network is trust on first use and can be downgraded to plain text by an attacker who can break the handshake. The list of keys trusted to sign closures is not bound to an authenticated identity, and with the key helper any holder of the secret can get signed paths trusted on every machine; the submitter's own upload is imported without a signature check; and the optional registry is unauthenticated. The repository's security statement describes these limits. The system is therefore suitable for machines whose owners trust one another, joined over networks they do not need to trust, which is a substantial change from before; it is not suitable for a cluster of mutually untrusting participants.

6.4 Future Work

- **Closing the security gaps.** Binding trusted signing keys to the identities that authenticated a link, requiring TLS rather than preferring it, checking signatures on every import path, and authorisation finer than one shared secret.
- **A relay of last resort**, for the peer pairs that no direct path can join, without making

it the default data path.

- **Locality-aware placement.** Placement currently scores capacity only. Accounting for which node already holds the input data, as iDDS [18] does, would reduce transfer for repeated work.
- **Topology-aware placement.** Whole jobs are placed on scalar metrics, blind to network distance; link cost is measured only where work is divided. A graph-structured representation [24] would address this.
- **Submitter fault tolerance for parallel scripts.** A remote job can already be detached and collected later; a script whose parallel work is spread cannot yet outlive its submitter, and a training ring cannot yet outlive rank 0's machine.
- **A cluster-wide view**, merging every machine's events, and a web page for the same statistics.
- **Additional runtimes** beyond Python.
- **A controlled multi-machine evaluation**, repeating the hand measurements of Chapter 5 as a scripted experiment across separate machines and networks, and verifying the per-accelerator ranks on a machine with two GPUs.

Bibliography

[1] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, "Spark: Cluster computing with working sets," *Proc. 2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud)*, 2010.

[2] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan, and I. Stoica, "Ray: A distributed framework for emerging AI applications," *Proc. 13th USENIX OSDI*, 2018, pp. 561–577.

[3] D. P. Anderson, "BOINC: A system for public-resource computing and storage," *Proc. 5th IEEE/ACM Workshop on Grid Computing*, 2004, pp. 4–10.

[4] M. Rocklin, "Dask: Parallel computation with blocked algorithms and task scheduling," *Proc. 14th Python in Science Conference (SciPy)*, 2015, pp. 130–136.

[5] E. Dolstra, M. de Jonge, and E. Visser, "Nix: A safe and policy-free system for software deployment," *Proc. 18th LISA*, 2004, pp. 79–92.

[6] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," *Proc. 6th USENIX OSDI*, 2004, pp. 137–150.

[7] Open Container Initiative, "OCI Runtime Specification, v1.0.2," 2021. [Online]. Available: https://github.com/opencontainers/runtime-spec.

[8] Synadia Communications, "NATS: Cloud native messaging." [Online]. Available: https://nats.io.

[9] S. Sekigawa, C. Sasaki, and A. Tagami, "Web application-based WebAssembly container platform for extreme edge computing," *Proc. IEEE GLOBECOM*, 2023, pp. 3609–3614.

[10] M. N. Hoque and K. A. Harras, "WebAssembly for edge computing: Potential and challenges," *IEEE Commun. Standards Mag.*, vol. 6, no. 4, pp. 68–73, Dec. 2022.

[11] S. Kakati and M. Brorsson, "A cross-architecture evaluation of WebAssembly in the cloud-edge continuum," *Proc. IEEE/ACM CCGrid*, 2024, pp. 337–346.

[12] Y. Li and S. Fujita, "Design of Elixir-based edge server for responsive IoT applications," *Proc. IEEE CANDARW*, 2023.

[13] Y. Zhou and X. Chen, "A synergistic Elixir-EDA-MQTT framework for advanced smart transportation systems," *Computers*, vol. 16, no. 3, 2024.

[14] J. Bachmeier, V. Yussupov, J. Henß, and H. Koziolek, "WebAssembly with wasi-nn for edge machine learning inference: Experiences and lessons learned," *Proc. ECSA*, 2025, pp. 343–359.

[15] S. E. Khelifa, M. Bagaa, A. O. Messaoud, and A. Ksentini, "Case study of WebAssembly runtimes for AI applications on the edge," 2024.

[16] A. Bergström, "Programming models for failure-transparent distributed systems," M.S. thesis, KTH Royal Institute of Technology, Stockholm, 2025. [Online]. Available: https://www.diva-portal.org/smash/get/diva2:2013047/FULLTEXT01.pdf.

[17] A. Meurer, S. Hoßfeld, R. Gommers, and the Consortium for Python Data API Standards, "Python array API standard: Toward array interoperability in the scientific Python ecosystem," *Proc. SciPy*, 2023. [Online]. Available: https://proceedings.scipy.org/articles/gerudo-f2bc6f59-001.

[18] W. Guan, T. Maeno, F. Barreiro Megino, T. Wenaus, and A. Klimentov, "iDDS: Intelligent distributed dispatch and scheduling for workflow orchestration," arXiv:2510.02930, 2025. [Online]. Available: https://arxiv.org/abs/2510.02930.

[19] Y. Deng, Z. Chen, X. Chen, and Y. Fang, "Task offloading in multi-hop relay-aided multi-access edge computing," *IEEE Trans. Veh. Technol.*, vol. 72, no. 1, pp. 1372–1376, Jan. 2023.

[20] H. He, X. Yang, X. Mi, H. Shen, and X. Liao, "Multi-agent deep reinforcement learning based dynamic task offloading in a device-to-device mobile-edge computing network," *Sensors*, vol. 24, no. 16, 2024.

[21] D. Yong, R. Liu, X. Jia, and Y. Gu, "Joint optimization of multi-user partial offloading strategy and resource allocation strategy in D2D-enabled MEC," *Sensors*, vol. 23, no. 5, 2023.

[22] L. Chen, Y. Wang, H. Zhang, and J. Liu, "Energy-efficient collaborative task offloading in multi-access edge computing based on deep reinforcement learning," *Ad Hoc Networks*, vol. 169, 2025.

[23] C. Li, Q. Chen, M. Chen, Z. Su, Y. Ding, D. Lan, and A. Taherkordi, "Blockchain enabled task offloading based on edge cooperation in the digital twin vehicular edge network," *J. Cloud Comput.*, vol. 12, no. 120, 2023.

[24] Z. Zhao, J. Perazzone, G. Verma, and S. Segarra, "Congestion-aware distributed task offloading in wireless multi-hop networks using graph neural networks," arXiv:2312.02471, 2023.

[25] J. Iyengar and M. Thomson, "QUIC: A UDP-based multiplexed and secure transport," RFC 9000, Internet Engineering Task Force, May 2021.

[26] E. Rescorla, "The Transport Layer Security (TLS) protocol version 1.3," RFC 8446, Internet Engineering Task Force, Aug. 2018.

[27] M. Petit-Huguenin, G. Salgueiro, J. Rosenberg, D. Wing, R. Mahy, and P. Matthews, "Session Traversal Utilities for NAT (STUN)," RFC 8489, Internet Engineering Task Force, Feb. 2020.

[28] D. MacDonald and B. Lowekamp, "NAT behavior discovery using Session Traversal Utilities for NAT (STUN)," RFC 5780, Internet Engineering Task Force, May 2010.

[29] S. Cheshire and M. Krochmal, "NAT Port Mapping Protocol (NAT-PMP)," RFC 6886, Internet Engineering Task Force, Apr. 2013.

[30] D. Wing, S. Cheshire, M. Boucadair, R. Penno, and P. Selkirk, "Port Control Protocol (PCP)," RFC 6887, Internet Engineering Task Force, Apr. 2013.

[31] UPnP Forum, "Internet Gateway Device (IGD) V2.0," Standardized DCP, 2010.

[32] B. Ford, P. Srisuresh, and D. Kegel, "Peer-to-peer communication across network address translators," in *Proc. USENIX Annual Technical Conf.*, 2005, pp. 179–192.

[33] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, "High-speed high-security signatures," *J. Cryptographic Engineering*, vol. 2, no. 2, pp. 77–89, 2012.

[34] Y. Collet and M. Kucherawy, "Zstandard compression and the 'application/zstd' media type," RFC 8878, Internet Engineering Task Force, Feb. 2021.

[35] D. G. Thaler and C. V. Ravishankar, "Using name-based mappings to increase hit rates," *IEEE/ACM Trans. Networking*, vol. 6, no. 1, pp. 1–14, Feb. 1998.

[36] E. Rescorla, "Keying material exporters for Transport Layer Security (TLS)," RFC 5705, Internet Engineering Task Force, Mar. 2010.